MACHINE LEARNING BASED DIABETES CLASSIFICATION AND PREDICTION

A Major Project report submitted

in partial fulfillment of requirement for the award of degree

BACHELOR OF TECHNOLOGY

in

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

By

B. PREETHIKA (2203A51106)

M. AKHILA (2203A51119)

Under the Guidance of

Mr. L. GUNAKAR RAO

Assistant Professor, School of CS&AI.

SR UNIVERSITY

SR University, Ananthsagar, Warangal, Telagnana-506371

SR University

Ananthasagar, Warangal.

SR UNIVERSITY

CERTIFICATE

This is to certify that this project entitled MACHINE LEARNING BASED DIABETES CLASSIFICATION AND PREDICTION is the bonafied work carried out by M. AKHILA , B. PREETHIKA as a Major Project for the partial fulfillment to award the degree BACHELOR OF TECHNOLOGY in School of Computer Science and Artificial Intelligence during the academic year 2025-2026 under our guidance and Supervision.

Mr. L. Gunakar Rao
Associate Professor
SR University
Anathasagar, Warangal

Dr. Ch.Sandeep
Asso. Professor & Head,
School of CS&AI,
SR University
Ananthasagar, Warangal.

Reviewer-1
Name:
Designation:
Signature:

Reviewer-2
Name:
Designation:
Signature:

DECLARATION

We declare that the work presented in this project report is original and has been carried out in the Department of Computer Science and Engineering, Kakatiya Institute of Technology and Science, Warangal, Telangana, and to best of our knowledge it has not been submitted elsewhere for any degree.

B. Preethika [2203A51106]

M. Akhila [2203A51119]

ACKNOWLEDGEMENT

We owe an enormous debt of gratitude to our Capstone project guide **Mr. L. Gunakar Rao , Assistant Professor** as well as Head of the School of CS&AI , **Dr. Ch. Sandeep, Assosiate Professor** for guiding us from the beginning through the end of the Capstone Project with their intellectual advices and insightful suggestions. We truly value their consistent feedback on our progress, which was always constructive and encouraging and ultimately drove us to the right direction.

We express our thanks to project co-ordinators **Mr. Sallauddin Md, Asst. Prof., and Mr. R.Ashok Asst. Prof.** for their encouragement and support.

Finally, we express our thanks to all the teaching and non-teaching staff of the department for their suggestions and timely support.

ABSTRACT

Millions of people worldwide suffer from diabetes, a chronic condition that requires early diagnosis and effective management to prevent major consequences. Hyperglycemia, another name for elevated blood glucose, is a chronic metabolic disease known as diabetes mellitus. When the body cannot use the insulin it generates or cannot make enough of it, disease results. The hormone insulin,which the pancreas produces ,secretes, assisting cells in using glucose as fuel. This hormone is necessary to regulate blood sugar levels.

Machine learning has emerged as a powerful tool in the healthcare sector, offering advanced methods for accurate diabetes classification and prediction. In order to identify whether or not a patient has diabetes, machine learning algorithms are used to evaluate their health data, including blood glucose levels, age, BMI, and lifestyle factors. The proposed system incorporates basic cloud computing for project deployment for scalable processing, making it accessible to a large number of users. While reducing false positives and negatives, feature selection, model optimization, and data treatment increase prediction accuracy. In detecting diabetes risk patterns, the outcomes show the potential of machine learning models such as Random Forest, and XGBoost

TABLE OF CONTENTS

		Page No.
CERTIFICATE		**ii**
DECLARATION		**iii**
ACKNOWLEDGEMENT		**iv**
ABSTRACT		**v**
TABLE OF CONTENTS		**vi**
LIST OF FIGURES		**viii**
LIST OF TABLES		**ix**
CHAPTER 1	**INTRODUCTION**	**01**
	1.1 INTRODUCTION	01
	1.2 OBJECTIVES	02
	1.3 METHODOLOGY	02
CHAPTER 2	**LITERATURE SURVEY**	**05**
CHAPTER 3	**IMPLEMENTATION**	**07**
	3.1 ALGORITHM	07
	3.1.1 RANDOM FOREST ALGORITHM	07
	3.1.2 XGBOOST ALGORITHM	10
	3.1.3 LIGHTGBM ALGORITHM	15
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	17
	3.3 TECHNOLOGY DESCRIPTION	18
	3.4 FLOW CHART	21
	3.5 DESIGN FOR TRAIN AND TEST DATA	22
	3.6 UML DIAGRAMS OF THE IMPLEMENTATION	23
CHAPTER 4	**EXPERIMENTATION AND IMPLEMENTATION**	**25**
	4.1 EXPERMENTATION	25
	4.1.1 DATA COLLECTION	25

4.1.2 DATA PRACESSING 26
4.1.3 FEATURE SELECTION 26
4.1.4 MODEL SELECTION 26
4.1.5 MODEL TRAINING 26
4.1.6 MODEL EVALUATION 26
4.2 IMPLENTATION OF ALGORITHMS 27
CHAPTER 5 RESULTS 38
CHAPTER 6 CONCLUSION 43
CHAPTER 7 FUTURE SCOPE & ENHANCEMENTS 44
REFERENCES 46
PUBLISHED RESEARCH PAPER 48

LIST OF FIGURES

Fig. No.	Name of the Figure	Page No.
3.1	Random Forest Algorithm	10
3.2	Light GBM Algorithm	17
3.3	Flow chart for Models	21
3.4	Division of Train and Test Data	22
3.5	Use Case Diagram	23
3.6	System Architecture	24
5.1	Sample Training Dataset	39

LIST OF TABLES

Table No.	Name of the Table	Page No.
3.1	Hardware Requirements	17
3.2	Software Requirements	17

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Diabetes, often known as diabetes mellitus, is a chronic metabolic disease characterized by elevated blood glucose levels. Disease arises when the body is unable to use the insulin it produces or is unable to produce enough of it. The hormone insulin, which is secreted by the pancreas, makes it simpler for cells to absorb glucose for use as fuel. Blood sugar regulation depends on this hormone.

Sedentary lifestyles, bad diets, ageing populations, and urbanization have all played a part in the steady increase in the prevalence of diabetes over the past few decades. According to projections from the International Diabetes Federation (IDF), 463 million persons aged 20 to 79 worldwide have diabetes in 2019. If present trends continue, this number is predicted to rise to 700 million by 2045.

Diabetes can lead to serious health problems if it is not properly managed. These implications include cardiovascular disease, stroke, kidney disease, nerve damage (neuropathy), visual problems (retinopathy), foot problems, and an increased risk of infection. In addition to being a major cause of death each year, diabetes is the cause of millions of fatalities worldwide. The financial burden Diabetes has a significant financial impact on individuals, families, healthcare institutions, and society as a whole. Direct medical expenditures for care and treatment and indirect costs like missed employment, disability, and early mortality are the two categories of costs associated with diabetes. Global healthcare expenditures associated with diabetes exceeded USD 760 billion in 2019, according to IDF projections.

Diabetes prevention's primary objectives are to promote healthy lifestyles that include consistent exercise, a well-balanced diet, weight control, healthcare services, promote healthy lifestyle choices, and raise awareness of illness to the general public.

There are several types of diabetes, however the two main ones are:

<u>Diabetes type 1:</u>

Cause: Unintentional immune system attack and loss of the pancreatic beta cells that produce insulin.

Onset: It is typically diagnosed in children and young adults, with symptoms often appearing during early adolescence. Early detection is crucial for managing the condition effectively and preventing long-term complications.

Treatment: Insulin pumps or needles must be used to administer insulin continuously, helping to regulate blood sugar levels throughout the day..

Diabetes type 2:

Cause: Abnormal blood glucose levels arise when the body produces insufficient amounts of insulin or develops resistance to it.

Onset: Adults are more prone to experience it, while children and teenagers can also get it.

Treatment: Start with dietary and activity adjustments to enhance lifestyle. Continue taking insulin or oral medicines as necessary.

1.2 OBJECTIVES

In our project, the objectives for Diabetes prediction are as follows:

- Develop a model to predict whether a person has diabetes or not based on input features.
- To streamline the process of model selection, hyperparameter tuning and performance evaluation with PIMA Indian Diabetes Dataset.
- To work on PIMA Indian Diabetes Dataset with the machine learning algorithms like Random Forest and XGBoost.
- Enhance model accuracy.

1.3 METHODOLOGY

1. Identification of Input and Output Parameters

- Define and select relevant input parameters, including patient demographic data (eg. age), clinical metrics (e.g., blood pressure, Insulin levels, Body Mass Index), lifestyle factors.
- Specify output parameters such as the likelihood or risk score for developing diabetes, including classification categories (e.g., diabetic or non-diabetic), and personalized recommendations for lifestyle changes or medical follow-up.

2. Data Collection and Preprocessing

- Compile information from pertinent sources, such as clinical databases and old patient records.
- To guarantee the dataset's quality and consistency, carry out data preprocessing procedures such as feature selection, data cleaning, normalization, and handling missing values.

3. Algorithm Selection and Comparison

- Identify a range of machine learning algorithms suitable for predictive modeling, such as XG Boost, Random Forest, or Light GBM
- Analyze the effectiveness of various algorithms using metrics like F1 score, recall, specificity, accuracy, and precision.
- Select the best-suited algorithm based on comparative analysis.

4. Selection of Software and Hardware Requirements

- Choose the software environment (e.g., Python, a cloud-based platform) and libraries (e.g., scikit-learn, xg boost) necessary for implementation.
- Determine the hardware resources required, such as computing power and storage, based on data size and computational demands.

5. Algorithm Implementation

- Implement the selected algorithm with the preprocessed dataset.
- Optimize model parameters using techniques like Randomized Search CV to enhance model performance

6. Data Analysis and Evaluation

- Analyze the results from the model implementation, interpreting key insights from performance metrics.
- Evaluate the importance of individual features in predicting Diabetes and assess the model's real-world applicability.

7. Report Preparation and Uniqueness Check

- Write a thorough report that includes the approach, findings, and interpretations.
- Verify the report's originality by underlining its distinctiveness with a plagiarism check.
- This structured approach enables accurate and insightful predictions for Diabetes, facilitating proactive healthcare interventions.

CHAPTER 2

LITERATUR SURVEY

2.1 OVERVIEW

Adeniyi et al. (2021) presented an IoMT-based wearable body sensors network for healthcare monitoring systems. The study explores the integration of the Internet of Medical Things (IoMT) with wearable sensors, enabling real-time health monitoring and improving healthcare delivery. Their approach highlights the potential of IoMT in chronic disease management and preventive healthcare [1].Rastogi and Bansal (2023) proposed a data mining-based diabetes prediction model. Their research demonstrates the utility of advanced data mining techniques in analyzing healthcare datasets to predict the likelihood of diabetes. The study emphasizes the importance of feature selection and algorithm optimization in improving predictive accuracy [2].

Pratap Singh et al. (2020) discussed the role of IoMT in orthopedics during the COVID-19 pandemic. They highlighted the challenges and applications of IoMT in maintaining healthcare services amidst restrictions, focusing on its utility in remote patient monitoring and teleconsultations [3].Larabi- Marie-Sainte et al. (2019) conducted a comprehensive review of current diabetes prediction techniques, presenting a case study that evaluates the effectiveness of machine learning models. Their work provides valuable insights into feature extraction and algorithmic performance in diabetes prediction [4].

Komi et al. (2017) explored the application of data mining methods in diabetes prediction. Their study discussed various techniques, emphasizing their capability to handle large datasets and improve diagnostic accuracy through advanced analytics [5].Ramanujam et al. (2020) developed a multilingual decision support system for early diabetes detection. Targeting rural Indian populations, their system utilized machine learning techniques to provide accessible and culturally sensitive healthcare solutions [6].

Maan et al. (2020) investigated the application of machine learning in diabetes prognostication. Their work highlighted the adaptability of different algorithms in predicting diabetes, emphasizing the significance of data preprocessing and algorithm tuning [7].Samant and Agarwal (2018a) applied machine learning techniques to diagnose diabetes using iris images. This innovative approach introduced a novel biometric method for medical diagnostics,

showcasing its potential for non-invasive screening [8]. In a subsequent study, Samant and Agarwal (2018b) performed a comparative analysis of classification algorithms for diabetes diagnosis using iris images. They evaluated various models, identifying key factors influencing classification accuracy [9].

Quinlan (1996) discussed decision tree classifiers, providing foundational insights into their development and application in medical diagnosis. His work underscored the importance of decision tree algorithms in predictive analytics and their interpretability in healthcare contexts [10].Saxena (2017) elaborated on the workings of decision tree algorithms, offering a detailed explanation of their construction and decision-making process. This resource serves as a valuable guide for implementing decision trees in diabetes prediction models [11].

Diabetes risk is predicted by predictive models using both history and current data. Neural networks, ensemble approaches, and logistic regression have all been used to forecast the risk of diabetes. Numerous research stress how crucial feature selection and preprocessing are to enhancing model performance. To determine the most important characteristics impacting diabetes risk, methods such as Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE) are employed. Preprocessing procedures like handling missing data and normalization are also emphasized as being essential to guaranteeing reliable forecasts. The literature regularly discusses cloud computing as a means of handling these data streams. Edge computing is becoming more popular as a solution to cloud computing's latency and privacy issues. In order to provide real-time predictions and lessen reliance on centralized systems, research is looking into implementing machine learning algorithms closer to data sour

CHAPTER 3

IMPLEMENTATION

3.1 ALGORITHM

The Algorithms used in our implementation are

- Random Forest
- XG Boost
- LightGBM

3.1.1 RANDOM FOREST ALGORITHM

Multiple decision trees are used in the Random Forest ensemble learning technique to produce a strong predictive model for both regression and classification tasks. After separately building multiple decision trees, each trained on a distinct portion of the training data, the method averages the predictions for tasks involving regression or uses a majority vote for tasks involving classification. Random Forest reduces the variation and overfitting frequently observed in individual decision trees by combining the output of several trees, producing a model with high stability and predictive power.

The Random Forest ensemble learning technique can be applied to classification and regression problems. Because of its exceptional accuracy and adaptability, this method is frequently utilized in practice to handle complex data.

Key Components of Random Forest:

Decision Trees: During training, the Random Forest ensemble learning technique generates a number of decision trees.

Random Subsampling: A random subset of the data is used to train each tree in the forest. This procedure is known as bootstrap sampling.

Randomly Selected Features: A random selection of features is considered for splitting at every decision tree node. This promotes the ensemble's diversity and makes it easier for the trees to decorrelate.

Averaging: In categorization jobs, the final forecast is decided by a majority vote of the trees. The average of each tree's predictions is the final prediction for regression issues.

Random Forest algorithm

a. Use the training dataset as input.
b. Make a decision tree with bootstrap sampling for each tree and random feature selection.
c. Group forecasting (classification).
d. Calculate the outcome.

How Random Forest Works

The approach starts by employing a method known as "bagging" (Bootstrap Aggregating) to create several decision trees. A random sample of the training data with replacement is used to train each tree in bagging; certain samples may be chosen more than once, while others might not be. A second layer of randomization is added when Random Forest creates each tree by choosing a subset of features at random at each split. This further lowers the correlation between trees and enhances generalization. Random Forest is a robust model appropriate for a variety of data sources since it combines these separate trees to minimize mistakes brought on by overfitting to particular patterns in individual trees.

During training, Random Forest builds a large number of decision trees and outputs the mean forecast for regression tasks or the mode of the trees' predictions for classification tasks. A random subset of the training data—created using a procedure known as bootstrapping, in which samples are drawn with replacement—is used to build each tree. In order to find the optimal split, a random subset of features is chosen at each tree node, adding even more variation to the trees. By lowering the correlation between individual trees, this feature selection technique improves the performance of the entire model. By combining the predictions from every tree, the final model successfully uses the wisdom of the community to increase robustness and accuracy.

Advantages of Random Forest

- **Reduce Overfitting**: Random Forest lowers the chance of overfitting, particularly when compared to individual decision trees, by averaging the predictions from numerous uncorrelated trees.
- **Feature Importance Insights**: By calculating the contribution of each feature to lowering the prediction error, Random Forest provides information on feature relevance. Finding the most important elements in complicated data sets may be made easier with this knowledge.

- **High Accuracy and Robustness**: Random Forest generally achieves high accuracy due to its ensemble nature, making it resilient to noise and outliers in the data.
- **Works Well with Missing Data**: Random Forest is more adaptable when working with partial datasets since it can efficiently accommodate missing values by utilizing surrogate splits or averaging predictions over numerous trees.
- **Handles Large Datasets with High Dimensionality**: Random Forest is capable of managing large datasets and high-dimensional feature spaces, making it suitable for tasks like image and text classification.

Applications and Comparisons to Other Algorithms

Random Forest is widely used across different industries, from healthcare for disease prediction and risk assessment to finance for credit scoring and fraud detection. In marketing, it can be applied to segment customers and predict consumer behavior. When compared to other ensemble techniques like Gradient Boosting, Random Forest typically requires less hyperparameter tuning and is less prone to overfitting in smaller datasets. However, Gradient Boosting methods (e.g., XGBoost, LightGBM) often yield higher accuracy on complex tasks, albeit with a higher computational cost. Random Forest's ability to provide quick, reliable predictions with minimal tuning makes it a popular choice for many real-world applications where simplicity and accuracy are both desired.

METHODOLOGY FOR RANDOM FOREST ALGORITHM

Pseudo Code for Random Forest Algorithm

```
Algorithm 1: Pseudo code for the random forest algorithm

To generate c classifiers:
for i = 1 to c do
    - Randomly sample the training dataset D with replacement to create a subset D_i.
    - Create a root node N_i that contains D_i.
    - Call BuildTree(N_i).
end for

BuildTree(N):
if N contains instances of only one class then
    - Exit and return (no further splitting is required).
else
    - Randomly select x% of the available features from N.
    - Choose the feature F that provides the best split based on the highest information gain.
    - Split N into f child nodes N_1, N_2, ..., N_f, where F has f possible values F_1, F_2, ..., F_f.

    for i = 1 to f do
        - Assign the corresponding subset D_i (instances matching F_i) to node N_i.
        - Recursively call BuildTree(N_i).
    end for
end if
```

Fig. 3.1 Random Forest Algorithm

3.1.2 XGBOOST ALGORITHM

The gradient boosting decision tree technique is the foundation of XGBoost, or "Extreme Gradient Boosting," a potent and incredibly effective machine learning framework. In predicting tasks requiring huge, complicated datasets, it frequently outperforms other algorithms due to its speed and performance. By adding optimizations like tree pruning, regularization to avoid overfitting, and parallel processing capabilities, XGBoost improves on conventional boosting techniques and is appropriate for both regression and classification tasks. Because of its capacity to produce precise and trustworthy findings, it is therefore widely used in many different disciplines. Extreme Gradient Boosting, or XGBoost, is a sophisticated gradient boosting method that optimizes performance and efficiency by including a number of cutting-edge features. One of its main features

is the application of a novel goal function that prevents overfitting and controls model complexity by combining the loss function with a regularization term. When the model is at risk of capturing noise instead of the underlying signal, or when dealing with high-dimensional data, this regularization is especially helpful.

Additionally, XGBoost has an extremely effective implementation that makes use of distributed computing and parallel processing, which enables it to scale well with big datasets. A method known as "block structure," which allows the algorithm to analyze data in chunks and greatly accelerate processing, is used to attain this efficiency. Additionally, XGBoost has an extensive collection of hyperparameters that can be adjusted to maximize model performance, making it extremely adaptable to various tasks and datasets.

XGBoost is widely used in business applications spanning from finance to healthcare and beyond, and it has become extremely popular in machine learning competitions like those held on Kaggle due to its excellent performance and versatility.

Working of XGBoost Algorithm:

A potent machine learning technique that expands on the ideas of gradient boosting is called Extreme Gradient Boosting, or XGBoost. This is an explanation of how XGBoost functions:

- **Boosting Framework:** The foundation of XGBoost is the boosting framework, which builds a strong learner by combining the predictions of multiple weak learners (usually decision trees). The goal is to gradually add models that fix the mistakes created by earlier models.
- **Initialization:** The procedure starts with an initial prediction, which is frequently a uniform probability for classification tasks or the mean of the target variable for regression assignments. The ensemble uses this as its starting point.
- **Iterative Model Building:** XGBoost uses an iterative process to construct trees. It concentrates on the residuals (errors) from the prior model in each iteration. In order to determine how to modify the predictions in order to minimize the loss, the algorithm computes the gradient (first derivative) and the hessian (second derivative) of the loss function with regard to the predictions.
- **Tree Construction:** To predict the residuals, a new decision tree is built for every iteration. A greedy algorithm that chooses the best splits based on the loss function's reduction

is used to build the tree. To manage the model's complexity and avoid overfitting XGBoost employs a regularized objective function that consists of the loss function and regularization term.

- **Regularization:** The objective function of XGBoost uses the L1 (Lasso) and L2 (Ridge) regularization approaches. This can improve generalization on unknown data by discouraging complex models and promoting simpler ones.
- **Learning Rate:** Each tree's predictions are subjected to a learning rate, commonly referred to as the shrinkage parameter. By scaling each tree's contribution, this parameter improves stability and enables more progressive model updates.
- **Managing Missing data:** A built-in method for managing missing data is present in XGBoost. It can make predictions without the need for imputation because it learns during training which way to go when it comes across missing data.
- **Parallel Processing:** XGBoost is made to be quick and efficient. By enabling parallel data processing through the use of a block structure, it greatly accelerates the computation of the optimal splits during tree construction.
- **Final Prediction:** The predictions from each tree are combined to create the final prediction once a predetermined number of trees have been constructed.
- **Hyperparameter Tuning:** The number of trees, maximum tree depth, learning rate, and regularization parameters are just a few of the many hyperparameters that may be adjusted with XGBoost to maximize model performance. Because of its adaptability, practitioners can customize the model for certain applications and datasets.

Advantages of XGBoost

- **Optimized Performance:** XGBoost outperforms several other gradient boosting frameworks due to its high speed and efficiency design. Training times are accelerated by its optimizations, which include pre-sorted data, caching, and distributed computing.
- **Regularization**: L1 (Lasso) and L2 (Ridge) regularization are features of XGBoost that help manage model complexity, lower the chance of overfitting, and enhance generalization on fresh data. Because of this, it is particularly useful for intricate models with plenty of elements.

- **Handling of Missing Data**: One of XGBoost's distinguishing advantages is its ability to manage missing values internally, allowing it to do calculations even when data points are incomplete, hence simplifying preparation.
- **Feature Importance Insights**: XGBoost gives extensive information on feature relevance, allowing data scientists to determine which variables have the greatest impact on model predictions. This interpretability capability is especially valuable in areas such as healthcare and finance, where understanding the primary elements driving decisions is crucial.

Applications and Comparisons with Other Algorithms

XGBoost is extremely adaptable and has been used successfully in fields such as healthcare, finance, marketing, and environmental studies. For example, in financial modelling, it is commonly used for credit scoring and fraud detection, which require great precision. In predictive health analytics, XGBoost's ability to handle complicated, high-dimensional data makes it excellent for identifying disease risk factors. When compared to other gradient boosting techniques such as LightGBM, XGBoost's level-wise tree building method is more stable on smaller datasets, however LightGBM may be faster on very big datasets. Despite this, XGBoost is still the favoured choice for its interpretability, precision, and solid performance across a wide range of application scenarios.

METHODOLOGY FOR XGBOOST ALGORITHM

Every Algorithm has its own method of performing its task. The methodology for XGBoost

Input: Training set: $\{(x_i, y_i)\}_{i=1}^{n}$

A differentiable loss function: L(y,F(x))

Number of weak learners: M

Learning rate: α\alpha

Algorithm:

a) **Initialize the model:**

Start with a constant value

$$f\hat{}((0))(x) = arg\min_{0} \sum_{i=1}^{N} L(y_i, 0).$$

b) **For m=1 to M:**

Perform the following steps:

- **Compute the gradients and Hessians (if applicable):**

Gradients (first-order derivative):

$$g^m(x_i) = [\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}]_{f(x)=f^{m-1}(x)}$$

Hessians (second-order derivative, used in advanced variants like XGBoost):

- **Fit a weak learner :**

Train the weak learner (e.g., decision tree) using the pseudo-residuals $g^m(x_i)$ and optionally $h^m(x_i)$.

This step i nvolves solving the optimization problem:

$$\emptyset^m = arg\min_{\emptyset C^{\emptyset}} \sum_{i=1}^{N} \frac{1}{2} h^m(x_i) \left[\emptyset(x_i) - \frac{g^m(x_i)}{h^m(x_i)}\right]^2$$

The result is the weak learner's prediction: $\emptyset^m(x)$.

c) **Update the model:**

Update the model by subtracting the contribution of the weak learner:

$$f^{\hat{}(m)}(x) = f^{\hat{}(m-1)}(x) - \propto \emptyset^m(x)$$

d) Output the final model:

After MM iterations, the final model is:

$$\hat{f}(x) = \hat{f}^{(M)}(x) = \sum_{m=0}^{M} \hat{f}^{m}(x).$$

3.1.3 LIGHTGBM (GRADIENT BOOSTING MACHINE) ALGORITHM

LightGBM is a distributed, high-speed gradient boosting system made for machine learning applications like regression, ranking, and classification that require accuracy and efficiency. LightGBM employs an original leaf-wise tree growth technique, in contrast to existing boosting algorithms that usually increase trees by levels. Instead of constructing trees level by level, this method enables LightGBM to choose the leaf that minimizes the greatest loss at each split. When compared to level-wise methods, this leaf-wise development frequently produces more precise results, improving performance without sacrificing economy. LightGBM's speed and lighter computational load are denoted by the "Light."

Advantages of LightGBM

- **High Efficiency and Faster Training**: LightGBM is a histogram-based approach that groups continuous feature values into discrete bins, which greatly speeds up training and increases its computational efficiency.
- **Less Memory Usage**: LightGBM uses less memory by dividing continuous values into discrete bins, which makes it appropriate for big datasets.
- **Improved Accuracy:** LightGBM can construct more intricate trees because to its leaf-wise development methodology, which frequently leads to improved accuracy. Setting a maximum tree depth can help control overfitting, albeit it can occasionally result from this.
- **Managing Big Datasets:** Unlike other boosting frameworks like XGBoost, LightGBM maintains good performance with shorter training times, making it ideal for handling huge datasets.
- **Supports Parallel Learning**: LightGBM offers parallel training capabilities, which further accelerates training times.

Disadvantages of LightGBM

- **Prone to Overfitting**: LightGBM's leaf-wise splitting approach occasionally leads to overfitting, especially when used with smaller datasets or unmonitored.
- **Takes Time to Tune for Complex Models:** In order to prevent underfitting or overfitting, LightGBM may need to be carefully adjusted for best results, particularly when working with complex models.

METHODOLOGY FOR LIGHTGBM ALGORITHM

Pseudo Code for LightGBM Algorithm (Paraphrased)

```
Algorithm 1: The LightGBM Algorithm

Input:
- Training data: D = {(x1, y1), (x2, y2), ..., (xN, yN)}, where xi ∈ X, X ⊆ R, yi ∈ {-1, +1}.
- Loss function: L(y, θ(x)); Iterations: M.
- Sampling ratios: a (large gradient data) and b (small gradient data).

Steps:
1. Combine mutually exclusive features using Exclusive Feature Bundling (EFB) method.
2. Initialize θ0(x) = arg min_θ Σ L(yi, θ(x)).
3. For m = 1 to M do:
    3.1. Compute absolute gradients:
         ri = |∂L(yi, θ(x))/∂θ(x)| evaluated at θ(x) = θm-1(x), i ∈ {1, ..., N}.
    3.2. Perform Gradient-based One-Side Sampling (GOSS):
         - Select top a × len(D) samples with large gradients.
         - Randomly sample b × len(D) samples with small gradients.
         - Form subset D' = A + B.
    3.3. Compute information gains for feature splits:
         Vj(d) = (1/n) [(Σxi∈A(ri) + (1-a)/b Σxi∈B(ri))^2 / ri(d)] + ...
    3.4. Find the best split using Vj(d) and grow a decision tree θm(x) on D'.
    3.5. Update: θm(x) = θm-1(x) + θm(x').
4. End for.

Output: Final model θ(x) = θM(x).
```

Fig. 3.2 Light GBM Algorithm

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The software requirements for implementing the machine learning-based diabetes classification and prediction include a range of programming tools, including machine learning libraries and data management solutions. Python happens to be the most required programming language due to extensive libraries such as Scikit-learn for NumPy, Pandas, and other machine learning algorithms for handling data manipulation and Seaborn or Matplotlib for visualization of Data. This integrated approach not only gives correct predictions but also allows real-time monitoring along with a user- friendly presentation of the result, which plays an important role in healthcare applications.

Functional Requirements include: Data preprocessing Machine learning algorithm implementations like SVM, XGBoost, and Random Forest Evaluations of the system performance, based on precision, recall, accuracy, and specificity. The system will have an easy- to-use interface to input and output data with the possibility of retraining with updated datasets; the prediction reports will be detailed.

HARDWARE REQUIREMENTS:

Processor	Core i3 processor
RAM	2GB
Hard Disk Capacity	50GB

Table 3.1 Hardware Requirements

SOFTWARE REQUIREMENTS

OS	Windows 7 versions
Language	Python
IDE	Visual studio code/ Jupyter or other IDEs

Table 3.2 Software Requirements

3.3 TECHNOLOGY DESCRIPTION

PYTHON

Python is a general-purpose superior and flexible language reputed to be easy to write and understand. Guido Van Rossum developed this programming language, and it comes into the market in the year 1991. Python emphasizes clean syntaxes and uses English-like words to minimize complexity as contrasted with other programming languages.

- Python, like PHP and PERL, executes code line-by-line without prior compilation.
- Python's interactive feature allows for real-time code testing and debugging.
- Python supports object-oriented programming, which encapsulates logic into objects.
- Python's easy syntax makes it very simple for newcomers to learn and use for a variety of applications, including text processing and gaming.

Python Features

- Python's syntax is easy to read and write, making it ideal for students and professionals. Its simplicity reduces the learning curve and enhances productivity.
- Python implements code line by line, abridging debugging and allowing interactive programming through tools like IDLE and Jupyter Notebook.
- Variable types are determined at runtime, allowing flexibility and reducing the need for explicit declarations.
- Python is platform-independent, meaning code can run on multiple operating systems like Linux, Macros, and Windows without alterations.
- Python embraces a thorough standard library that supports a wide range of functionalities such as file handling, math operations, and web development.
- Python accommodates procedural, object-oriented, and functional programming styles, catering to diverse developer needs.
- Python boasts an extensive group of third-party libraries, including TensorFlow, NumPy, Pandas, Flask, and Django, which simplify tasks in data science, AI, web development, and more.
- Python can handle small scripts as well as large, complex applications, making it suitable for various domains like web development, automation, and scientific computing.

- Python has an active global community that provides extensive documentation, tutorials, and forums, ensuring developers have access to resources and help.
- Python is compatible with various IDEs like PyCharm, VS Code, and Spyder, which offer tools for debugging, testing, and code management.
- Python processes memory allocation and garbage gathering automatically, reducing the developer's workload and minimizing memory leaks.

Advanced Features

- Supports functional, structured, and object-oriented programming paradigms.
- Provides dynamic data types and automated garbage collection.
- Easily integrates with programming languages like C, C++, Java, and CORBA.

HTML

HTML (HyperText Markup Language) is the most widely used markup language for web page creation. It gives online material organization through a system of tags and attributes. Headings, paragraphs, links, images, lists, and other components make up HTML pages and define the structure and content of a webpage. These parts are included in angle brackets; some of them require opening and closing tags, while others close by themselves. An element's attributes provide additional information about it, such as its ID, class, or source URL.

The declaration that indicates the HTML version is followed by the element that includes the entire page. The element is composed of two main parts. The section contains meta-data about the content, such as the document title, character encoding, and connections to other resources like stylesheets and scripts. The element contains the viewable content of the webpage, which consists of text, images, and other media.

HTML is an essential part of web development, and JavaScript is commonly used in conjunction with HTML for interactivity and CSS (Cascading Style Sheets) for styling. It is necessary to create well- organized, accessible web content and is supported by all current web browser.

DJANGO

An open-source, high-level web framework called Django is used to create web applications with the Python programming language. By offering a comprehensive suite of tools and features that simplify basic web development chores, it was created to make the development process simpler and quicker.

Architecture:

Model-View-Template (MVT), which is comparable to Model-View-Controller (MVC), is the architectural pattern that Django adheres to.

The MVT architecture's constituent parts are:

- Model: Shows the application's business logic and data structure. Models are Python classes that are used to communicate with databases.
- The logic that handles user requests and provides answers is contained in the view. Views are in charge of pulling information from the model and using templates to render it.
- Template: Specifies the application's presentation layer. HTML files called templates can include dynamic material that is produced by the views.

Key Features:

Django has many built-in features that speed up development. These include:

- **Admin Interface:** To manage application data, Django automatically creates a robust and adaptable admin interface that is accessible via a web browser.
- **Object-Relational Mapping:** ORM is a feature of Django that enables programmers to communicate with databases using Python objects rather than unformatted SQL queries. This abstraction improves code readability and streamlines database operations.
- **URL Routing:** Django gives developers the ability to create clear, intuitive URLs for their apps with its adaptable URL routing system.
- **Authentication and Authorization:** User accounts, permissions, and access control are simple to handle with Django's integrated authentication and authorization features.
- **Form Handling:** Django streamlines form handling, which is crucial for online applications and includes user input validation, rendering, and processing.
- **Security Features:** Django comes with built-in defenses against typical online threats including cross-site scripting (XSS), SQL injection, and cross-site request forgery (CSRF).
- **Localization & Internationalization:** Django makes it simpler to create apps for a worldwide user base by supporting a variety of languages and time zones.
- **Scalability:** By adding more servers, Django may be expanded horizontally to accommodate applications with high traffic volumes.

3.4 FLOW CHART

The accompanying advances are to be maintained for the information to be passed from one high level to the base level that is displayed in Figure 3.3.

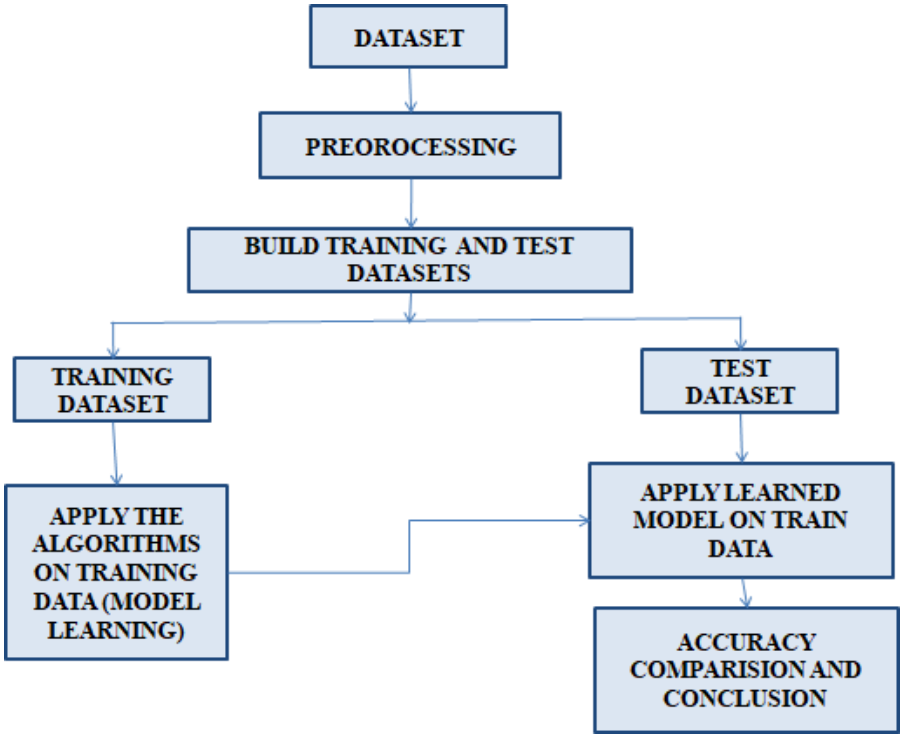


Fig. 3.3 Flow chart for Models

The flowchart depicts a machine learning workflow, starting with data preprocessing and splitting into training and testing datasets. The training dataset is used to build the model, while the test dataset evaluates its performance. The process concludes with accuracy comparison and drawing insights for model effectiveness.

3.5 DESIGN FOR TRAIN AND TEST DATA

In each Machine Learning approach, the informative indexes used for the expectation must be segregated into preparing and testing, as shown in Fig. 3.4.

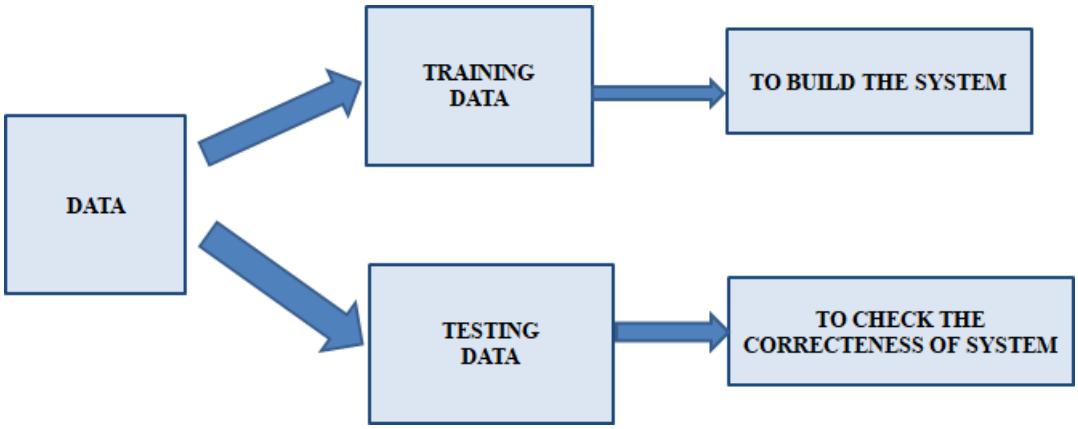


Fig. 3.4 Division of Train and Test Data

This flowchart summarizes the division of data in a machine learning workflow. The dataset is split into two parts:

1. **Training Data**: Used to build and train the machine learning model by learning patterns and relationships within the data.
2. **Testing Data**: Used to evaluate and validate the correctness and performance of the trained model on unseen data.

3.6 UML DIAGRAMS OF THE IMPLEMENTATION

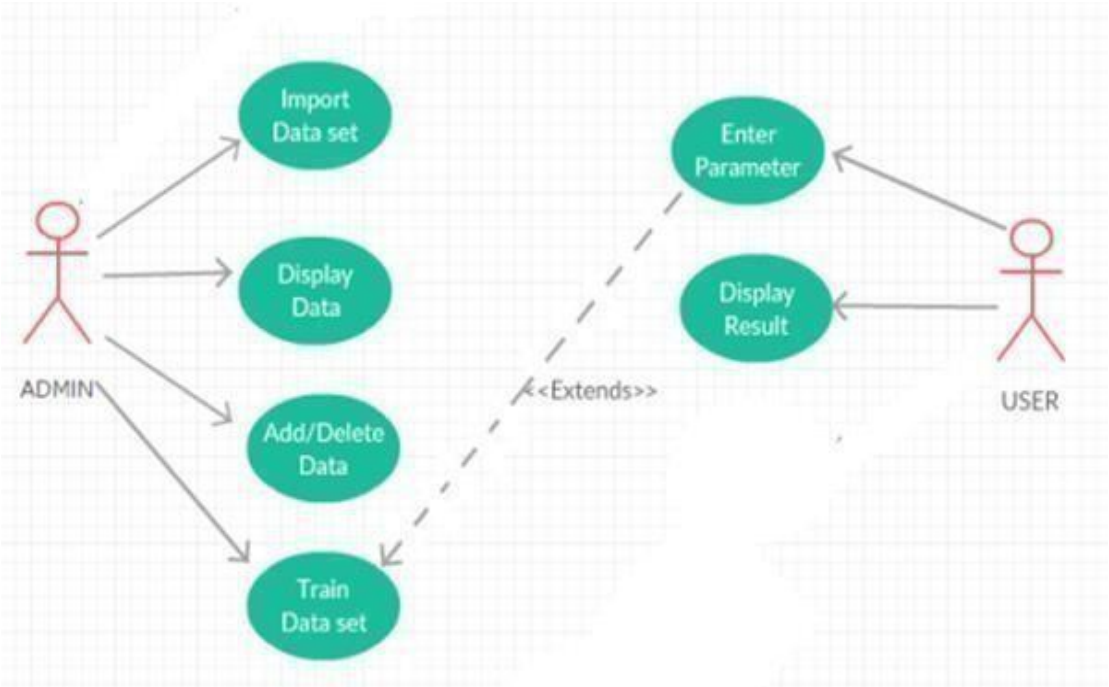


Fig 3.5 Use Case Diagram

The Use Case Diagram depicts two actors: Admin and User. The Admin manages backend functions like importing, modifying, and training datasets, while the User interacts with the system by entering parameters and viewing results. Key processes include dataset management and result display, with extended functionality for data modification.

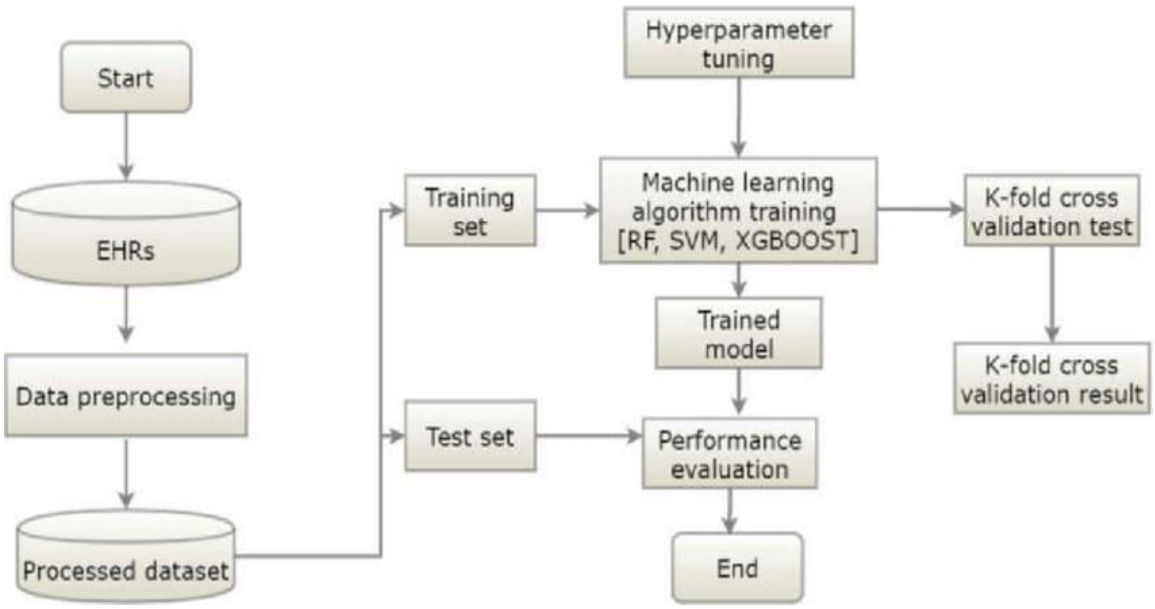


Fig 3.6 System Architecture

The diagram outlines a machine learning workflow using EHR data. It involves data preprocessing, splitting into training and test sets, hyperparameter tuning, training models (RF, SVM, XGBoost), performing K-fold cross-validation, and evaluating model performance to finalize the process.

CHAPTER 4

EXPERIMENTATION AND IMPLEMENTATION

4.1 EXPERIMENTATION

In this experiment, we analyse algorithms using the PIMA dataset, which contains a variety of clinical and demographic characteristics related to diabetes. The dataset is preprocessed and split into testing and training subsets for assessment. The algorithms used in our project are:

- Random Froest
- XGBoost
- LightGBM

Here is a brief description of the steps leading to model evaluation for Diabetes Prediction and classification

4.1.1 Data Collection:

Data for a diabetes prediction project is typically collected from several sources, including laboratory test results, electronic health records (EHRs), demographic report, and lifestyle data. Demographic elements like gender, ethnicity and age play a significant role in diabetes risk assessment, as do lifestyle behaviors like physical activity levels, dietary habits, smoking, and alcohol consumption. A family history of diabetes and, in women, a history of gestational diabetes further increase the likelihood of developing the condition. By combining these features, the dataset gives a thorough foundation for training machine learning models to predict diabetes risk accurately and efficiently.

The PIMA Indian Diabetes Dataset is one of the publicly accessible datasets from which the system gathers data.Data preparation is carried out to increase model efficiency. This include removing outliers, changing numerical characteristics, addressing missing values with imputation techniques, and getting rid of duplicate entries. Additionally, only the most pertinent features for prediction are kept by using feature selection techniques like Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE).

4.1.2 Data Preprocessing:

- **Handling Missing Values:** Missing data is imputed using statistical measures to ensure the dataset is complete.
- **Data Normalization:** Continuous features are scaled to ensure uniformity and improve algorithm performance.

4.1.3 Feature Selection:

- Important predictors are identified to focus the model on relevant attributes. The Recursive Feature Elimination (RFE) or feature importance scores from XGBoost are used.
- Selected features include critical markers like blood glucose levels, insulin levels and body mass index (BMI)

4.1.4 Model Selection:

- Algorithms such as LightGBM, Random Forest, XGBoost are chosen based on their suitability for medical prediction tasks.
- These models are optimized for accuracy, interpretability, and scalability, depending on project goals.

4.1.5 Model Training:

- **Hyperparameter Tuning:** Methods like Randomized Search CV or Grid Search CV are employed to fine-tune parameters like learning rates or maximum depth.
- **Validation:** Cross-validation is conducted to assess the model's performance on unseen data, ensuring it generalizes well.

4.1.6 Model Evaluation:

- Models are evaluated based on accuracy, precision, recall, F1-score, and AUC-ROC.
- Multiple algorithms are examined to identify the best model for predicting Diabetes.

4.2 IMPLEMENTATION OF ALGORITHMS

```
import numpy as np import pandas as pd
import statsmodels.api as sm import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import scale, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.metrics import confusion_matrix, accuracy_score, mean_squared_error, r2_score,
roc_auc_score, roc_curve, classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from lightgbm import LGBMClassifier
from sklearn.model_selection import KFold
import warnings
warnings.simplefilter(action = "ignore")
df= pd.read_csv("E:\diabetes.csv")
df.head()
df.shape
 df.info()
df.describe([0.10,0.25,0.50,0.75,0.90,0.95,0.99]).T
df["Outcome"].value_counts()*100/len(df)
df.Outcome.value_counts()
df["Age"].hist(edgecolor = "black");
print("Max Age: " + str(df["Age"].max()) + " Min Age: " + str(df["Age"].min()))
fig, ax = plt.subplots(4,2, figsize=(16,16))
```

```
sns.distplot(df.Age, bins = 20, ax=ax[0,0])
sns.distplot(df.Pregnancies, bins = 20, ax=ax[0,1])
sns.distplot(df.Glucose, bins = 20, ax=ax[1,0])
sns.distplot(df.BloodPressure, bins = 20, ax=ax[1,1])
sns.distplot(df.SkinThickness, bins = 20, ax=ax[2,0])
sns.distplot(df.Insulin, bins = 20, ax=ax[2,1])
sns.distplot(df.DiabetesPedigreeFunction, bins = 20, ax=ax[3,0])
sns.distplot(df.BMI,bins=20,ax=ax[3,1])
df.groupby("Outcome").agg({"Pregnancies":"mean"})
df.groupby("Outcome").agg({"Age":"mean"})
df.groupby("Outcome").agg({"Insulin": "mean"})
f,ax=plt.subplots(1,2,figsize=(18,8))
df['Outcome'].value_counts().plot.pie(explode=[0,0.1],autopct='%1.1f%%',ax=ax[0],shadow=Tru
e)
ax[0].set_title('target')
ax[0].set_ylabel('')
sns.countplot('Outcome',data=df,ax=ax[1])
ax[1].set_title('Outcome')
plt.show()
f, ax = plt.subplots(figsize= [20,15])
sns.heatmap(df.corr(), annot=True, fmt=".2f", ax=ax, cmap = "magma" )
ax.set_title("Correlation Matrix", fontsize=20)
plt.show()
df[['Glucose','BloodPressure','SkinThickness','Insulin','BMI']] =
df[['Glucose','BloodPressure','SkinThickness','Insulin','BMI']].replace(0,np.NaN)
pip install missingno
import missingno as msno
msno.bar(df);
```

```
def median_target(var):
temp = df[df[var].notnull()]
temp = temp[[var, 'Outcome']].groupby(['Outcome'])[[var]].median().reset_index() return temp
columns = df.columns
columns = columns.drop("Outcome") for i in columns:
median_target(i)
df.loc[(df['Outcome'] == 0 ) & (df[i].isnull()), i] = median_target(i)[i][0]
df.loc[(df['Outcome'] == 1 ) & (df[i].isnull()), i] = median_target(i)[i][1] for feature in df:
Q1 = df[feature].quantile(0.25) Q3 = df[feature].quantile(0.75)
IQR = Q3-Q1
lower = Q1- 1.5*IQR upper = Q3 + 1.5*IQR
if df[(df[feature] > upper)].any(axis=None):
    print(feature,"yes")
else:
    print(feature, "no")
import seaborn as sns
sns.boxplot(x = df["Insulin"]);
Q1 = df.Insulin.quantile(0.25)
Q3 = df.Insulin.quantile(0.75)
IQR = Q3-Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR
df.loc[df["Insulin"] > upper,"Insulin"] = upper
from sklearn.neighbors import LocalOutlierFactor
 lof =LocalOutlierFactor(n_neighbors= 10)
lof.fit_predict(df)
df_scores = lof.negative_outlier_factor_
np.sort(df_scores)[0:30]
threshold = np.sort(df_scores)[7]
```

```
threshold
NewBMI = pd.Series(["Underweight", "Normal", "Overweight", "Obesity 1", "Obesity 2",
"Obesity 3"], dtype = "category")
df["NewBMI"] = NewBMI
df.loc[df["BMI"] < 18.5, "NewBMI"] = NewBMI[0]
df.loc[(df["BMI"] > 18.5) & (df["BMI"] <= 24.9), "NewBMI"] = NewBMI[1]
df.loc[(df["BMI"] > 24.9) & (df["BMI"] <= 29.9), "NewBMI"] = NewBMI[2]
df.loc[(df["BMI"] > 29.9) & (df["BMI"] <= 34.9), "NewBMI"] = NewBMI[3]
df.loc[(df["BMI"] > 34.9) & (df["BMI"] <= 39.9), "NewBMI"] = NewBMI[4]
df.loc[df["BMI"] > 39.9 ,"NewBMI"] = NewBMI[5]
df.head()
def set_insulin(row):
    if row["Insulin"] >= 16 and row["Insulin"] <= 166:
        return "Normal"
    else:
        return "Abnormal"
# The operation performed was added to the dataframe.
df = df.assign(NewInsulinScore=df.apply(set_insulin, axis=1))

df.head()
NewGlucose = pd.Series(["Low", "Normal", "Overweight", "Secret", "High"], dtype =
"category")
df["NewGlucose"] = NewGlucose
df.loc[df["Glucose"] <= 70, "NewGlucose"] = NewGlucose[0]
df.loc[(df["Glucose"] > 70) & (df["Glucose"] <= 99), "NewGlucose"] = NewGlucose[1]
df.loc[(df["Glucose"] > 99) & (df["Glucose"] <= 126), "NewGlucose"] = NewGlucose[2]
df.loc[df["Glucose"] > 126 ,"NewGlucose"] = NewGlucose[3]
```

```
df.head()
df = pd.get_dummies(df, columns =["NewBMI","NewInsulinScore", "NewGlucose"], drop_first = True)
df.head()
categorical_df = df[['NewBMI_Obesity 1','NewBMI_Obesity 2', 'NewBMI_Obesity 3', 'NewBMI_Overweight','NewBMI_Underweight','NewInsulinScore_Normal','NewGlucose_Low' ,'NewGlucose_Normal', 'NewGlucose_Overweight', 'NewGlucose_Secret']]
categorical_df.head()
y = df["Outcome"]
X = df.drop(["Outcome",'NewBMI_Obesity 1','NewBMI_Obesity 2', 'NewBMI_Obesity 3', 'NewBMI_Overweight','NewBMI_Underweight','NewInsulinScore_Normal','NewGlucose_Low' ,'NewGlucose_Normal', 'NewGlucose_Overweight', 'NewGlucose_Secret'], axis = 1)
cols = X.columns
index = X.index
X.head()
from sklearn.preprocessing import RobustScaler
transformer = RobustScaler().fit(X)
X = transformer.transform(X)
X = pd.DataFrame(X, columns = cols, index = index)
X.head()
# Import necessary libraries
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from lightgbm import LGBMClassifier
```

```
# Define models
models = []
models.append(('LR', LogisticRegression(random_state = 12345)))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier(random_state = 12345)))
models.append(('RF', RandomForestClassifier(random_state = 12345)))
models.append(('SVM', SVC(gamma='auto', random_state = 12345)))
models.append(('XGB', GradientBoostingClassifier(random_state = 12345)))

# evaluate each model in turn
results = []
names = []
print(models)

for name, model in models:
    kfold = KFold(n_splits = 10)
    cv_results = cross_val_score(model, X, y, cv = 10, scoring= "accuracy")
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)

# boxplot algorithm comparison
fig = plt.figure(figsize=(15,10))
fig.suptitle('Algorithm Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()
```

```
models = []

models.append(('RF', RandomForestClassifier(random_state = 12345, max_depth = 8, max_features = 7, min_samples_split = 2, n_estimators = 500)))
models.append(('XGB', GradientBoostingClassifier(random_state = 12345, learning_rate = 0.1, max_depth = 5, min_samples_split = 0.1, n_estimators = 100, subsample = 1.0)))

# evaluate each model in turn
results = []
names = []
for name, model in models:
    kfold = KFold(n_splits = 10)
    cv_results = cross_val_score(model, X, y, cv = 10, scoring= "accuracy")
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(),cv_results.std())
    print(msg)

# boxplot algorithm comparison
fig = plt.figure(figsize=(15,10))
fig.suptitle('Algorithm Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler, PowerTransformer
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report
from imblearn.over_sampling import SMOTE
                                        #PROPOSED MODEL
# Load the dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI',
'DiabetesPedigreeFunction', 'Age', 'Outcome']
data = pd.read_csv(url, names=columns)

# Handle missing or zero values by replacing them with the median
for col in ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']:
    data[col].replace(0, data[col].median(), inplace=True)

# Split into features (X) and target (y)
X = data.drop('Outcome', axis=1)
y = data['Outcome']

# Handle class imbalance using SMOTE
smote = SMOTE(random_state=123)
X, y = smote.fit_resample(X, y)

# Apply Power Transformation to make features more Gaussian-like
pt = PowerTransformer(method='yeo-johnson', standardize=True)
X = pt.fit_transform(X)

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=123,
stratify=y)

# Hyperparameter tuning using GridSearchCV for Random Forest
 rf_params = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}
rf_grid = GridSearchCV(RandomForestClassifier(random_state=123), rf_params, cv=3,
scoring='accuracy', n_jobs=-1, verbose=1)
rf_grid.fit(X_train, y_train)
rf_model = rf_grid.best_estimator_

# Hyperparameter tuning using GridSearchCV for XGBoost
xgb_params = {
   'n_estimators': [100, 200, 300],
   'max_depth': [3, 5, 7],
   'learning_rate': [0.01, 0.05, 0.1],
   'subsample': [0.8, 1.0],
   'colsample_bytree': [0.8, 1.0]
}
xgb_grid = GridSearchCV(XGBClassifier(eval_metric='logloss', random_state=123),
xgb_params, cv=3, scoring='accuracy', n_jobs=-1, verbose=1)
xgb_grid.fit(X_train, y_train)
xgb_model = xgb_grid.best_estimator_
```

```
# Evaluate models
rf_pred = rf_model.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_pred)
print(f"Random Forest Accuracy (Improved): {rf_accuracy * 100:.2f}%")
print(classification_report(y_test, rf_pred))

xgb_pred = xgb_model.predict(X_test)
xgb_accuracy = accuracy_score(y_test, xgb_pred)
print(f"XGBoost Accuracy (Improved): {xgb_accuracy * 100:.2f}%")
print(classification_report(y_test, xgb_pred))

# Take user input for predictions
print("\nEnter the patient's information to predict diabetes:")
try:
    pregnancies = float(input("Number of pregnancies: "))
    glucose = float(input("Glucose level: "))
    blood_pressure = float(input("Blood pressure (mm Hg): "))
    skin_thickness = float(input("Skin thickness (mm): "))
    insulin = float(input("Insulin level (IU/mL): "))
    bmi = float(input("Body Mass Index (BMI): "))
    dpf = float(input("Diabetes Pedigree Function: "))
    age = float(input("Age: "))

    # Combine inputs into a list
    user_data = [[pregnancies, glucose, blood_pressure, skin_thickness, insulin, bmi, dpf, age]]
    user_data = pt.transform(user_data)  # Apply power transform
    user_data_scaled = scaler.transform(user_data)  # Standardize the input
```

```
# Predict using Random Forest
rf_prediction = rf_model.predict(user_data_scaled)
print("\nRandom Forest Prediction (0 = No, 1 = Yes):", rf_prediction[0])

# Predict using XGBoost
xgb_prediction = xgb_model.predict(user_data_scaled)
print("XGBoost Prediction (0 = No, 1 = Yes):", xgb_prediction[0])

except ValueError:
    print("\nError: Please enter valid numerical inputs.")
```

CHAPTER 5
RESULTS

SAMPLE TRAINING DATASET

The Pima Indian Diabetes Dataset is a well-known dataset that is used to predict the onset of diabetes in women of Pima Indian ancestry. Eight diagnostic parameters are included in the 768 cases: age, BMI, hypertension (BP), skin thickness, insulin levels, glucose levels, diabetes pedigree function (a measure of family history), and pregnancies. Outcome, target variable, is a binary class that determines whether someone has diabetes (1) or not (0). This dataset is often used in machine learning as a benchmark, particularly for those with tasks of binary classification.

ANALYSIS OF ALGORITHMS:

The metrics used to analyze models for Diabetes prediction include accuracy, precision, recall, F1-score, and Area Under the Receiver Operating Characteristic Curve (AUC-ROC). Although accuracy provides a broad measure of correctness, precision and recall are important measures to ensure that true positives are captured while

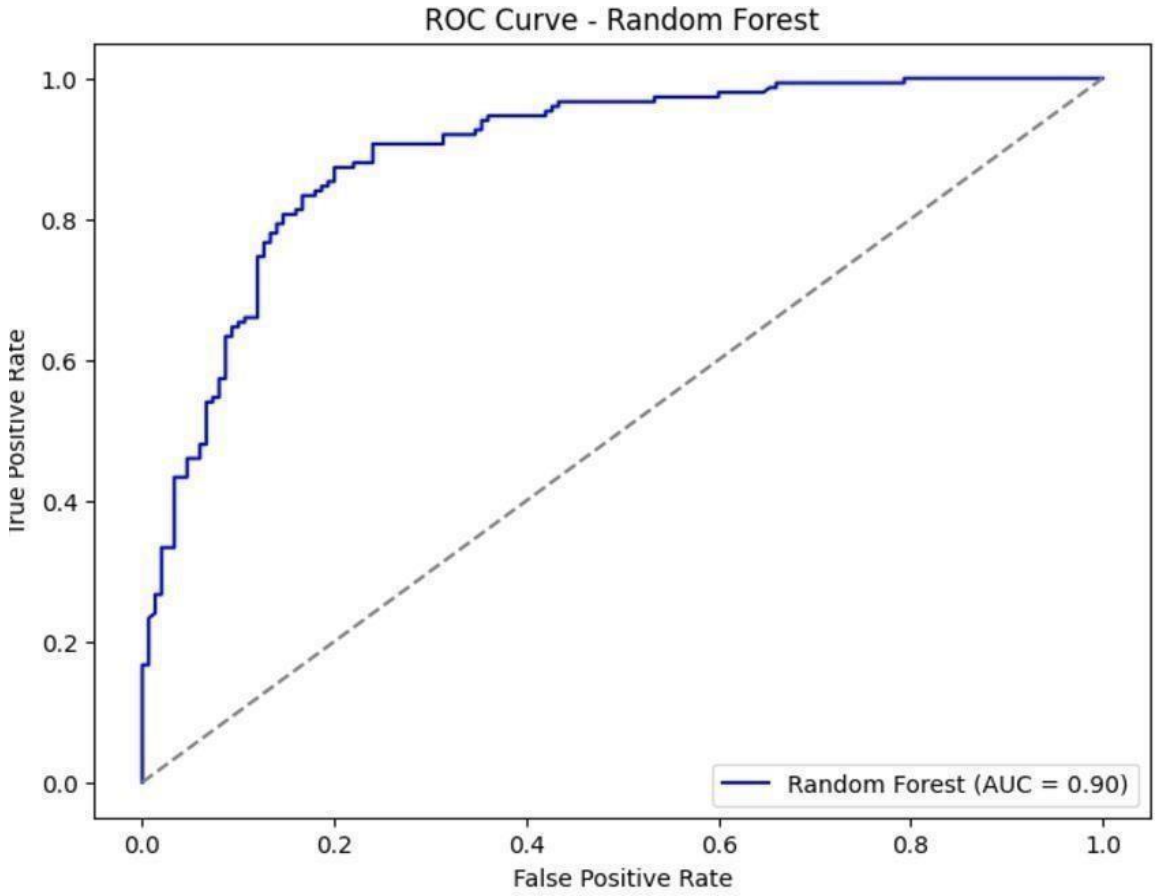
Based on the parameters considered such as accuracy and other parameters the best suited algorithm for our data set is "XGBoost" Algorithm.

	A	B	C	D	E	F	G	H	I
1	Pregnancie	Glucose	BloodPres	SkinThickn	Insulin	BMI	DiabetesP	Age	Outcome
2	6	148	72	35	0	33.6	0.627	50	1
3	1	85	66	29	0	26.6	0.351	31	0
4	8	183	64	0	0	23.3	0.672	32	1
5	1	89	66	23	94	28.1	0.167	21	0
6	0	137	40	35	168	43.1	2.288	33	1
7	5	116	74	0	0	25.6	0.201	30	0
8	3	78	50	32	88	31	0.248	26	1
9	10	115	0	0	0	35.3	0.134	29	0
10	2	197	70	45	543	30.5	0.158	53	1
11	8	125	96	0	0	0	0.232	54	1
12	4	110	92	0	0	37.6	0.191	30	0
13	10	168	74	0	0	38	0.537	34	1
14	10	139	80	0	0	27.1	1.441	57	0
15	1	189	60	23	846	30.1	0.398	59	1
16	5	166	72	19	175	25.8	0.587	51	1
17	7	100	0	0	0	30	0.484	32	1
18	0	118	84	47	230	45.8	0.551	31	1
19	7	107	74	0	0	29.6	0.254	31	1
20	1	103	30	38	83	43.3	0.183	33	0
21	1	115	70	30	96	34.6	0.529	32	1
22	3	126	88	41	235	39.3	0.704	27	0
23	8	99	84	0	0	35.4	0.388	50	0
24	7	196	90	0	0	39.8	0.451	41	1
25	9	119	80	35	0	29	0.263	29	1
26	11	143	94	33	146	36.6	0.254	51	1
27	10	125	70	26	115	31.1	0.205	41	1
28	7	147	76	0	0	39.4	0.257	43	1
29	1	97	66	15	140	23.2	0.487	22	0
30	13	145	82	19	110	22.2	0.245	57	0
31	5	117	92	0	0	34.1	0.337	38	0

Fig.5.1 Sample Training Dataset

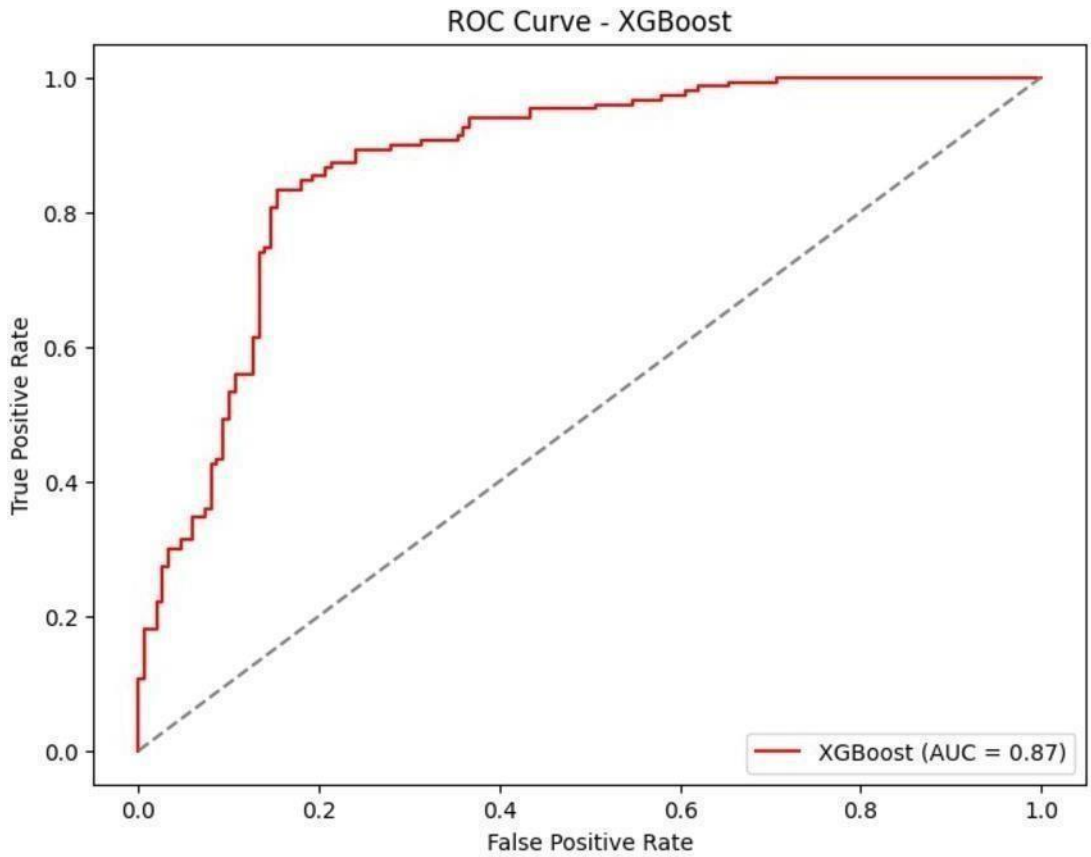
Accuracy For Random Forest

```
Random Forest Accuracy: 83.00%
              precision    recall  f1-score   support

           0       0.86      0.79      0.82       150
           1       0.80      0.87      0.84       150

    accuracy                           0.83       300
   macro avg       0.83      0.83      0.83       300
weighted avg       0.83      0.83      0.83       300

Random Forest AUC: 0.90
```

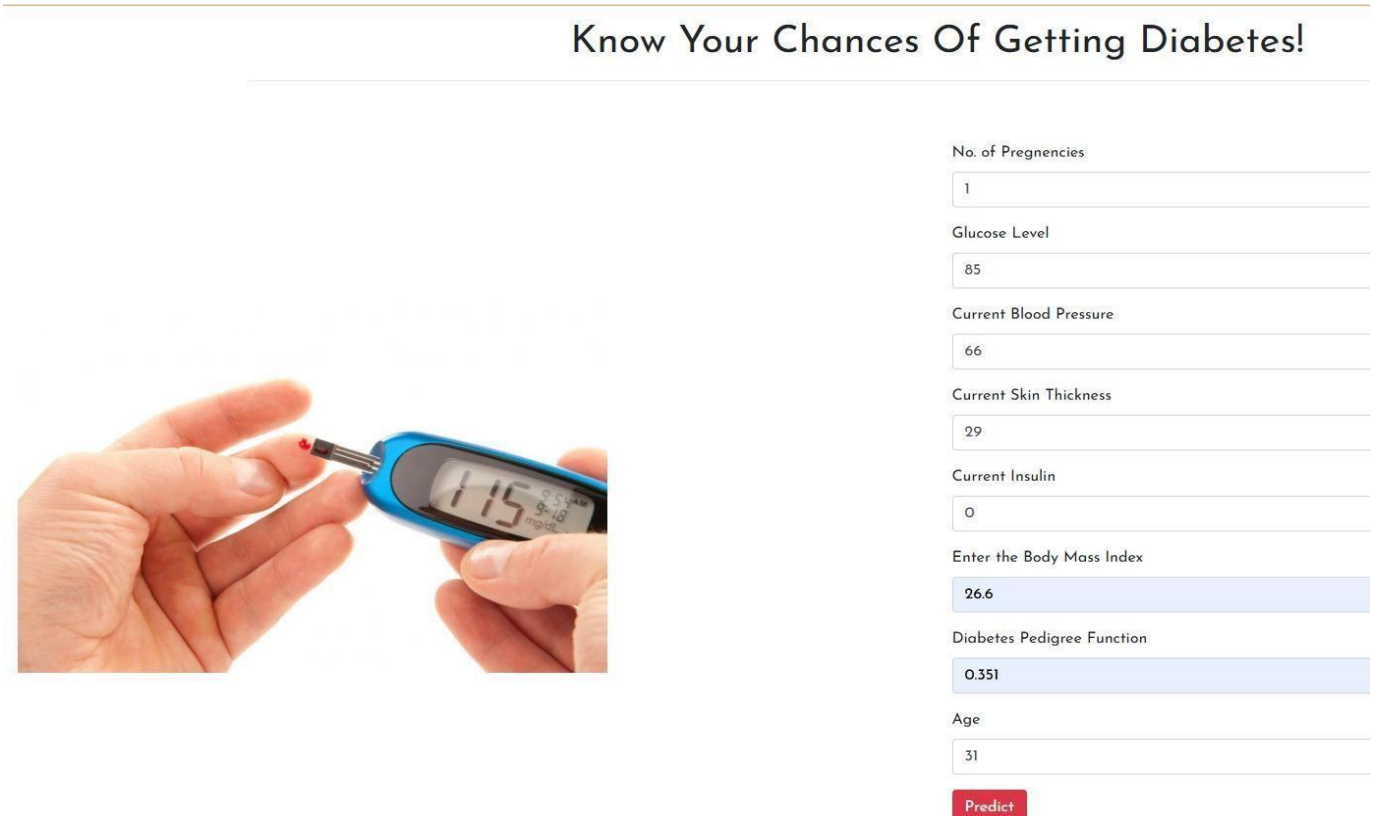


Accuracy For XGBoost

```
XGBoost Accuracy: 82.00%
              precision    recall  f1-score   support

           0       0.86      0.76      0.81       150
           1       0.79      0.88      0.83       150

    accuracy                           0.82       300
   macro avg       0.82      0.82      0.82       300
weighted avg       0.82      0.82      0.82       300

XGBoost AUC: 0.87
```



Sample input



Sample output

CHAPTER 6
CONCLUSION

CONCLUSION

In conclusion, machine learning techniques like XG Boost, Random Forest, and Light GBM are robust solutions to diabetes classification and prediction as they make the most of their advanced capabilities in dealing with complex datasets. XG Boost offers high accuracy in terms of its gradient boosting framework. Random Forest excels in handling data distributions with different kinds of distribution and has fewer chances of overfitting. Light GBM excels at processing large- scale data, with relatively low computational requirements. It will further be possible to increase predictive accuracy and reliability of the diagnosis by integrating these models with real-time data collection systems.

The suggested machine learning-based diabetes classification and prediction system significantly increases the precision, effectiveness, and accessibility of diabetes diagnosis in comparison to conventional techniques. The system generates faster, more accurate, and data-driven predictions by using state-of-the-art machine learning algorithms like XGBoost and Random Forest and by deploying the project in the cloud (wherever Python can access it). Predictive analytics, real-time alerts, and automated decision-making work together to enhance patient outcomes, reduce the risk of complications, and enable early detection.

Through feature selection, data preprocessing, and hyperparameter tuning, the model achieves high precision while minimizing false positives and negatives. Patients and healthcare professionals can easily and smoothly engage with the system by using a Flask-based web application.

CHAPTER 7

FUTURE SCOPE &ENHANCEMENTS

In this project, we utilized static data for training and testing the algorithms to predict the risk of Diabetes. In the future, the scope of this project can be expanded in the following ways:

a. **Wearable device integration**: In the future, wearable medical technology (such as smartwatches and continuous glucose monitors) may be connected with diabetes prediction algorithms.

b. **Big Data Utilization for Model Training**: Training machine learning models using extensive healthcare data (such as EHRs, genetic data, and population health datasets).

c. **Using a Cloud-Based Model**: For scalable and adaptable access, diabetes prediction models can be deployed on cloud platforms like AWS, Google Cloud, or Azure.

d. **Federated Learning and Continuous Learning**: Using federated learning to train models on data spread across several devices or hospitals without sending sensitive data.

e. **Predicting Diabetes Risk Automatically**: Using machine learning models based on demographic, lifestyle, and health aspects to develop automated solutions for early risk detection of diabetes.

f. **Predictive Analytics in the Cloud for Healthcare Providers**: Offering cloud-based solutions for diabetes management and prediction in real time to clinics and hospitals.

g. **Security and Privacy of Data in Cloud-based Machine Learning Models**: Protecting sensitive patient data on cloud-based systems by implementing secure access protocols and cutting-edge encryption techniques.

h. **Integration with EHR Systems:** For smooth data transmission and automated predictions during regular examinations, diabetes prediction models can be integrated with current EHR systems.

i. **Extended Observation and Post-Treatment Forecasting**: Predicting diabetes patients' post- treatment problems or recovery using machine learning models.

j. **Model Optimization in Environments with Limited Resources**: Improving machine learning models for implementation in low-resource settings, including remote medical facilities or clinics with constrained processing capacity.

k. **Adaptability of Models Across Cultures**: Creating diabetes prediction models that may be modified to accommodate various ethnic, regional, and cultural contexts.

l. **Using AI to Forecast Complications from Diabetes:** Creating models to forecast consequences (e.g., neuropathy, kidney failure) as well as the development of diabetes.

m. **Predictive Maintenance in Real Time for Medical Equipment:** Using cloud-based analytics & ML models to forecast maintenance requirements for diabetes management equipment (such as insulin pumps and glucometers).

n. **Integration of Web and Mobile Applications Across Platforms:** Creating web and mobile apps with cloud-hosted machine learning models to provide users with recommendations and predictions in real time.

REFERENCES

[1]Adeniyi, E. A., Ogundokun, R. O., & Awotunde, J. B. (2021). IoMT-Based Wearable Body Sensors Network Healthcare Monitoring System. In G. Marques, A. K. Bhoi, V. H. C. de Albuquerque, & H. KS (Eds.), IoT in Healthcare and Ambient Assisted Living (Vol. 933, pp. 103–121). Springer Singapore. https://doi.org/10.1007/978-981-15-9897-5_6 R. Rastogi, M. Bansal, Diabetes prediction model using data mining techniques (2023)

[2]Divya, K., Sirohi, A., Pande, S., & Malik, R. (2021). An IoMT Assisted Heart Disease Diagnostic System Using Machine Learning Techniques. In A. E. Hassanien, A. Khamparia, D. Gupta, K. Shankar, & A. Slowik (Eds.), Cognitive Internet of Medical Things for Smart Healthcare (Vol. 311, pp. 145–161). Springer International Publishing. https://doi.org/10.1007/978-3-030-55833-8_9

[3]Pratap Singh, R., Javaid, M., Haleem, A., Vaishya, R., & Ali, S. (2020). Internet of Medical Things (IoMT) for orthopaedic in COVID-19 pandemic: Roles, challenges, and applications. Journal of Clinical Orthopaedics and Trauma, 11(4), 713–717. https://doi.org/10.1016/j.jcot.2020.05.011

[4]Larabi-Marie-Sainte, S., Aburahmah, L., Almohaini, R. and Saba, T. (2019). 'Current Techniques for Diabetes Prediction: Review and Case Study', Applied Sciences, vol. 9, no. 21, pp. 4604.

[5]Komi, M., Jun Li, Yongxin Zhai, Xianguo Zhang, 2017. Application of data mining methods in diabetes prediction, in: 2017 2nd International Conference on Image, Vision and Computing (ICIVC), Chengdu, https://doi.org/10.1109/ICIVC.2017.7984706 China, pp. 1006–1010.

Ramanujam, E., Chandrakumar, T., Thivyadharsine, K.T., Varsha, D., 2020. A Multilingual Decision Support System for early detection of diabetes using Machine Learning approach: Case study for Rural Indian people, in: 2020 Fifth International Conference on Research in Computational Intelligence and Communication Networks (ICRCICN). Bangalore, India, pp.

17–21. https://doi.org/10.1109/ICRCICN50933.2020.9296187

[6] Maan, V., Vijaywargiya, J., Srivastava, M., 2020. Diabetes Prognostication – An Aptness of Machine Learning, in: 2020 International Conference on Emerging Trends in Communication, Control and Computing (ICONC3). Lakshmangarh, Sikar, India, pp. 1–5. https://doi.org/10.1109/ICONC345789.2020.9117465

[7] Samant, P., Agarwal, R., 2018a. Machine learning techniques for medical diagnosis of diabetes using iris images. Computer Methods and Programs in Biomedicine 157, 121 128. https://doi.org/10.1016/j.cmpb.2018.01.004

[8] Samant, P., Agarwal, R., 2018b. Comparative analysis of classification based algorithms for diabetes diagnosis using iris images. Journal of Medical Engineering & Technology 42, 35–42. https://doi.org/10.1080/03091902.2017.1412521

[9] Quinlan, J. R. (1996). Learning decision tree classifiers. ACM Computing Surveys (CSUR), 28(1), 71-72.

[10] Saxena, R. (2017). How Decision Tree Algorithm works. Available at: https://dataaspirant.com/2017/01/30/how-decision-tree-algorithm-works/ (accessed April 40).

RESEARCH PAPER
Machine Learning Based Diabetes Classification and Prediction

Mr. L. Gunakar Rao
Assisstant Professor
Department of CSE, SRU
Email:gunakarrao@gmail.com

M. Akhila
2203A51119, SRU
Email: mittapellyakhila71@gmail.com

B. Preethika
2203A51106, SRU
Email:preethikabongu@gmail.com

***Abstract*: A chronic illness that affects millions of people worldwide, diabetes must be diagnosed early and managed well to avoid serious complications. In the healthcare industry, machine learning has become a potent instrument, providing sophisticated techniques for precise diabetes classification and prediction. In this study, patients' health data, including blood glucose levels, age, BMI, and lifestyle characteristics, are analyzed using machine learning algorithms to determine if they have diabetes or not. For salable processing, the suggested system includes the basic cloud computing for the deployment of project so that it can be accessed by many people . Feature selection, model optimization, and data treatment improve prediction accuracy while lowering false positives and negatives. In detecting diabetes risk patterns, the outcomes show the potential of machine learning models such as Random Forest, and XGBoost.**

***KEYWORDS*: Diabetes, Machine Learning, Prediction, Classification, IoT, Cloud Computing, Data Preprocessing.**

I .INTRODUCTION

The chronic metabolic disease known as diabetes, or diabetes mellifluous, is typified by consistently elevated blood glucose levels. It happens because of either the pancreas's insufficient production of insulin or the body's incapacity to utilize the insulin it does produce efficiently. An essential hormone, insulin helps the body's cells absorb glucose so that it can be used as fuel. When insulin isn't working properly, glucose stays in the blood, which can cause major health issues. Maintaining general health depends on blood sugar management, and problems with this process can have serious long-term effects. [5][9].

Due to a number of causes, such as sedentary lifestyles, poor eating habits, rising obesity rates, urbanization, and aging populations, the prevalence of diabetes has

increased dramatically over the past few decades. Globally, 463 million people between the ages of 20 and 79 had diabetes in 2019, according to estimates from the International Diabetes Federation (IDF). If present trends continue, it is estimated that this number will riseto 700 million by 2045 [4][6]. The increasing rise in diabetes cases is a contributing factor to a global public health emergency that requires prompt treatment [3].

Diabetes is one of the leading causes of blindness, kidney failure, heart attacks, and lower limb amputations worldwide [7], and it has a significant negative impact on quality of life and places a heavy burden on healthcare systems. If left untreated or inadequately managed, diabetes can lead to multiple organ damage, including heart disease, stroke, chronic renal disease, nephropathy, foot ulcers that require amputation, and an elevated risk of infection [5][8]. In addition to the direct impact on health, diabetes has serious economic ramifications.

The financial burden associated with diabetes includes direct medical costs for treatment and hospitalization, as well as indirect costs such as loss of productivity, disability, and premature death [10][11]. According to the IDF, global healthcare expenditures related to diabetes exceeded USD 760 billion in 2019, a figure expected to rise further as the prevalence of the disease increases [2].

The basic goals of diabetes management and prevention are to maintain balanced diets, encourage regular physical activity, encourage better lives, ensure early diagnosis, and provide efficient treatment strategies [1][14]. Access to healthcare services and public knowledge are essential for stopping the disease's progression. Artificial intelligence (AI), machine learning models, and the Internet of Things (IoT) for real-time health monitoring are examples of technical and medical breakthroughs that are currently being used to improve diabetes management, classification, and prediction [3][12][19].

Type 1 diabetes is an autoimmune disease in which the body's immune system mistakenly attacks and destroys insulin-producing beta cells in the pancreas [15]. It is typically diagnosed in children, adolescents, or young adults, but it can develop at any age [9]. People with Type 1 diabetes need lifelong insulin therapy, which can be given through insulin injections or insulin pumps [8]. Without insulin, blood glucose levels can rise dangerously high, leading to diabetic acidosis (DKA), a condition that can be fatal [6].

Type 2 diabetes arises when the body either grows resistant to insulin or fails to produce enough insulin to maintain normal blood sugar levels [7]. It is more common in adults, particularly those over the age of 40, but has been increasingly documented in children and adolescents due to increased obesity rates and bad lifestyle behaviors [4]. Initially, weight reduction, dietary modifications, and increased physical activity can be used to manage Type 2 diabetes [1]. However, insulin therapy or oral medicines may be necessary when the illness worsens [13].

Other types of diabetes exist in addition to Type 1 and Type 2 diabetes. Gestational diabetes, which develops during pregnancy and raises the risk of Type 2 diabetes later in life [20], and secondary diabetes, which is brought on by other illnesses or the use of specific medications [17]. Early detection and continuous monitoring are essential for preventing complications and improving patient outcomes, particularly given the increasing burden of diabetes on international health systems[2][14].

Furthermore, integrating cloud computing, wearable technology made possible by the Internet of Things, and AI-driven healthcare solutions can revolutionize diabetes management by enabling proactive interventions and continuous health monitoring [12][21].

II. LITERATURE SURVEY

In order to improve diabetes prediction and monitoring, numerous studies have concentrated on combining cutting-edge technologies such as cloud computing, artificial intelligence (AI), big data analytics, and the Internet of Medical Things (IoMT). This section examines the body of research on machine learning models for diabetes prediction and categorization, emphasizing important techniques and developments in technology. An IoMT-based wearable body sensor network for healthcare monitoring was presented by Adeniyi et al. (2021), illustrating how real-time health tracking might enhance the management of chronic diseases. Researchers are looking into cutting-edge techniques for early diagnosis, classification, and management utilizing machine learning and data-driven approaches as a result of the increased prevalence of diabetes worldwide.According to their research, continuous glucose monitoring and more accurate and timely diabetes projections can be obtained by combining wearable sensors and cloud computing. Astrologist and Balsa (2023) also presented a data mining-based diabetes prediction model, making use of feature selection and algorithm development to improve prediction accuracy. Their findings highlights the need of pre-processing techniques for improved model performance, such as feature scaling, handling missing data, and selecting relevant biomarkers.

Pratap Singh and colleagues' 2020 study explored the application of IoMT in orthopedic healthcare during the COVID-19 pandemic, emphasizing its applications in AI-driven medicine, diagnostics, and remote monitoring. Their findings suggest that IoMT-based options could revolutionize the way chronic diseases are treated by reducing hospital stays while preserving continuous patient monitoring.

In order to provide accurate forecasts and customized health recommendations, their approach combined machine learning algorithms with an easy-to-use user interface, thereby addressing the accessibility issue in remote places.

The use of AI in diabetes diagnoses was further expanded by Maan et al. (2020), who investigated machine learning techniques for diabetes prognostication, comparing different algorithms such as XGBoost, LightGBM, and Decision Trees. According to their research, ensemble-based learning techniques significantly increase predictive accuracy, especially when working with datasets that are unbalanced. Samant and Agarwal (2018a) presented yet another novel approach, using iris image processing to diagnose diabetes. Their study examined the feasibility of using biometric-based screening to find early signs of diabetes as an alternative to traditional blood glucose testing.

In a follow-up study that examined classification algorithms, Mantissa and Agarwal (2018b) assessed the impact of several feature selection strategies on diabetes diagnosis using iris photos.

In addition to conducting a comprehensive examination into the development of decision tree models for

medicine, Palanquin (1996) provided basic insights into decision tree classifiers and their interpretability in predictive analytics. Building on this, Saxena (2017) offered a thorough process for creating and refining decision trees in order to implement categorization models for healthcare applications. These studies show how decision trees are being used more and more in medical diagnostics due to their transparency and interoperability.

Recent advances in machine learning have highlighted the importance of feature selection and preprocessing methods for diabetes prediction. By removing unnecessary or redundant features, preparatory stages like Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE) significantly increase model accuracy, according to numerous studies. It has also been crucial to improve model performance to handle imbalanced datasets using underselling and oversampling strategies, such as the Synthetic Minority Over-sampling Technique (SMOTE).

Another recent development in diabetes prediction is the integration of cloud and edge computing to offer real-time healthcare monitoring. Cloud computing has long been used to analyze and store large amounts of patient data, but concerns about latency and data privacy have led researchers to investigate edge computing options. By analyzing data closer to the source, like on wearables with Internet of Things capabilities, researchers seek to improve real-time diabetes management decision-making and speed up response times.

Federated learning is also gaining popularity as a possible privacy-preserving healthcare machine learning solution. Federated learning, as opposed to typical centralized learning models that need to transfer data to a central server, enables decentralized model training across several devices without exchanging raw patient data. This method helps predict diabetes since it allows hospitals and research groups to collaborate on model development while maintaining patient privacy.

The use of deep learning techniques, such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks, for diabetes classification and prediction is also highlighted in the research. Particularly when it comes to using retinal imaging to detect retinopathy in diabetic patients, deep learning models have proven to be incredibly adept at analyzing complex medical data. These tactics have the potential to transform non-invasive diabetes screening methods by doing away with the necessity for traditional blood tests.

In conclusion, recent studies demonstrate that machine learning techniques—from federated learning and deep learning to traditional classification algorithms—offer significant improvements in the classification and prediction of diabetes. Even if problems like data quality, model interpretability, and computational complexity still persist, ongoing advancements in cloud computing, IoT, and privacy-preserving AI are making it possible to create diabetes management solutions that are more efficient, scalable, and real-time. As machine learning develops, future research should focus on customized healthcare interventions that integrate real-time patient monitoring and predictive analytics for early diabetes detection and prevention.

III.EXISTING SYSTEM

In order to detect risk factors and diagnose diabetes, the traditional approach mostly relies on clinical testing,

manual data processing, and medical expertise [5][9]. Blood tests like the Oral Glucose Tolerance Test (OGTT), Hemoglobin A1c (HbA1c), and Fasting Blood Sugar (FBS) are commonly used to diagnose diabetes and assess glucose levels [3]. Despite their effectiveness, these methods are often time-consuming, invasive, and require frequent inspections to ensure proper disease management [7][14]. Additionally, the majority of conventional diagnostic techniques rely on static data, meaning they do not account for real-time changes in blood glucose levels or lifestyle-related risk factors that influence the development of diabetes [12][19].

In many healthcare settings, rule-based decision algorithms are used to analyze patient information in order to identify probable cases of diabetes [4][10]. These algorithms follow established medical norms and use historical patient data to classify individuals as either diabetes or non-diabetic [6]. However, these methods are not highly predictive and do not work well when working with large datasets or identifying complex patterns in patient data [13][21]. It is difficult to forecast diabetes risk based on lifestyle, genetic predisposition, and environmental factors since the existing approach lacks predictive analysis and real-time monitoring [11][20]. Many remain undiagnosed for extended periods of time, increasing the risk of severe consequences such as cardiovascular diseases, diabetes retinopathy, kidney failure, and neuropathy [16][23]. Without a continuous monitoring system, it is difficult to take preventative and early intervention measures, which are crucial for managing chronic conditions like diabetes [17].

Data fragmentation across many healthcare organizations is another drawback of the current system, which makes it challenging to integrate and conduct a thorough analysis of patient information [22]. Advanced analytics, machine learning models, and cloud-based healthcare solutions—all of which could greatly increase the accuracy of diabetes prediction and classification—are not supported by traditional databases [18][24]. Effective diabetes management is further hampered by privacy issues and a lack of interoperability across various healthcare platforms [25].

Disadvantages of Existing System:

-The use of IoT devices raises significant issues regarding the privacy of patient data. Unauthorized access to sensitive health information can lead to misuse or exploitation.

-Maintainig the vast amount of data generated by the IOT devices is difficult.

-Depending on IOT device may give the incorrect information if there are any technical issues

IV PROPOSED SYSTEM:

In order to overcome the shortcomings of conventional diagnostic techniques, the suggested methodology presents a machine learning-based method for diabetes classification and prediction [5][9]. The approach increases the accuracy and efficiency of diabetes risk identification by utilizing sophisticated machine learning techniques [7][12].The system applies feature selection, optimizes hyperparameters, and analyzes patient records using algorithms such as XGBoost, Random Forest.

In addition to machine learning, the system incorporates cloud computing to ensure that it can be accessed by any one .This encourages early intervention by providing real-time diabetes risk tracking for patients and healthcare providers and automated notifications for high-risk situations [8][17]. The suggested system offers continuous analysis, predictive insights, and tailored healthcare recommendations based on patient-specific data patterns, in contrast to traditional approaches that depend on repetitive testing [11][19]. This approach improves the efficacy, reliability, and accessibility of diabetes diagnosis and management through automated decision-making, real-time warnings, and enhanced scalability [20].

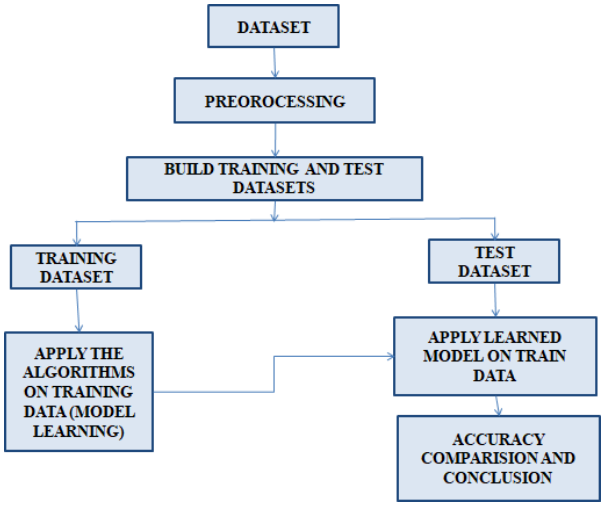


Fig.1. Flowchart for proposed models

PROPOSED SYSTEM ADVANTAGES

· Increased Accuracy — Machine learning algorithms improve forecast accuracy by analyzing complex patterns in patient data.

· Early Detection & Alerts: Predictive analytics assists in identifying high-risk individuals so that timely treatment and intervention can occur

-Manual Collection: Manual testing methods, such as fingerstick blood glucose tests, are often less expensive than continuous monitoring systems. This makes them more accessible for patients who may not have the financial means to invest in advanced technology.

IMPLEMENTATION AND METHODOLOGY

For efficient diabetes classification and prediction, the proposed system is put into practice utilizing a methodical methodology that blends machine learning algorithms, cloud-based data processing A variety of crucial input statistics are initially determined by the system, including age, BMI, blood pressure, blood sugar, insulin, number of pregnancies, and lifestyle factors. The result is a classification that shows if a person has diabetes. Following data collection from medical records, Internet of Things sensors, and publicly available datasets such as the PIMA Indian Diabetes Dataset, preprocessing techniques such as data cleaning, feature selection, normalization, and outlier treatment are employed to ensure high-qualityinput for the model.

The proposed solution takes a deliberate, systematic approach to ensure efficient diabetes classification and prediction through the use of cloud computing, IoT-based monitoring, and machine learning. Every stage is crucial to enhancing the system's accuracy, reliability, and real-time performance.

Step 1: Finding the Input and Output Parameters

Age, BMI, blood pressure, blood sugar, insulin, blood pressure, number of pregnancies, and lifestyle factors including nutrition and physical exercise are among the system's primary input requirements. These

characteristics have a significant impact on diabetes risk. The result of the data analysis is a categorization label that indicates whether or not a person has diabetes.

Step 2: Data Gathering and Preparation

The system collects information from the publicly available datasets such as the PIMA Indian Diabetes Dataset.To improve model efficiency, data preprocessing is done. This include eliminating duplicate entries, modifying numerical properties, handling missing numbers using imputation techniques, and eliminating outliers. Furthermore, by employing feature selection methods such as Principal Component Analysis (PCA) and Recursive Feature Elimination (RFE), only the most relevant features for prediction are retained.

Step 3: Selecting Algorithms and Training Models

Several machine learning techniques, including Random Forest and XGBoost are considered for categorization. The dataset is divided into 80% training data and 20% testing data in order to train and evaluate the models. To optimize model performance, hyperparameter adjustments are made using Grid Search CV and Randomized Search CV. The system also employs cross-validation (K-Fold CV) to ensure that the model generalizes effectively to unknown data and to reduce overfitting.

Step4: User Interface Development and Model Deployment. The improved machine learning model is then delivered via a Flask-based web application following training. The application allows users to input health parameters, which are then processed by an algorithm to provide instantaneous diabetes risk estimates. To ensure that patients and medical professionals can simply access it, an intuitive user interface is designed

Step 5: Performance Assessment and Enhancement

The final model is evaluated using metrics such as F1-score, AUC-ROC, accuracy, precision, and recall to confirm the system's effectiveness. The system's performance is continuouslytracked, allowing for future improvements through incremental learning and dataset expansion.

RESULTS AND DISCUSSION

	A	B	C	D	E	F	G	H	I
1	Pregnancie	Glucose	BloodPres	SkinThickn	Insulin	BMI	DiabetesPe	Age	Outcome
2	6	148	72	35	0	33.6	0.627	50	1
3	1	85	66	29	0	26.6	0.351	31	0
4	8	183	64	0	0	23.3	0.672	32	1
5	1	89	66	23	94	28.1	0.167	21	0
6	0	137	40	35	168	43.1	2.288	33	1
7	5	116	74	0	0	25.6	0.201	30	0
8	3	78	50	32	88	31	0.248	26	1
9	10	115	0	0	0	35.3	0.134	29	0
10	2	197	70	45	543	30.5	0.158	53	1
11	8	125	96	0	0	0	0.232	54	1
12	4	110	92	0	0	37.6	0.191	30	0
13	10	168	74	0	0	38	0.537	34	1
14	10	139	80	0	0	27.1	1.441	57	0
15	1	189	60	23	846	30.1	0.398	59	1
16	5	166	72	19	175	25.8	0.587	51	1
17	7	100	0	0	0	30	0.484	32	1
18	0	118	84	47	230	45.8	0.551	31	1
19	7	107	74	0	0	29.6	0.254	31	1
20	1	103	30	38	83	43.3	0.183	33	0
21	1	115	70	30	96	34.6	0.529	32	1
22	3	126	88	41	235	39.3	0.704	27	0
23	8	99	84	0	0	35.4	0.388	50	0
24	7	196	90	0	0	39.8	0.451	41	1
25	9	119	80	35	0	29	0.263	29	1
26	11	143	94	33	146	36.6	0.254	51	1
27	10	125	70	26	115	31.1	0.205	41	1
28	7	147	76	0	0	39.4	0.257	43	1
29	1	97	66	15	140	23.2	0.487	22	0
30	13	145	82	19	110	22.2	0.245	57	0
31	5	117	92	0	0	34.1	0.337	38	0

Fig. 2. Training Dataset

EXISTING SYSTEM ACCURACY

```
Naive Bayes Accuracy: 74.33%
              precision    recall  f1-score   support

           0       0.71      0.82      0.76       150
           1       0.79      0.67      0.72       150

    accuracy                           0.74       300
   macro avg       0.75      0.74      0.74       300
weighted avg       0.75      0.74      0.74       300
```

Fig.3 Naives Bayes Accuracy

```
Decision Tree Accuracy: 76.00%
              precision    recall  f1-score   support

           0       0.75      0.76      0.76       146
           1       0.77      0.76      0.76       154

    accuracy                           0.76       300
   macro avg       0.76      0.76      0.76       300
weighted avg       0.76      0.76      0.76       300
```

Fig. 4 Decision Tree Accuracy

PROPOSED SYSTEM ACCURACY

```
Random Forest Accuracy: 83.00%
              precision    recall  f1-score   support

           0       0.86      0.79      0.82       150
           1       0.80      0.87      0.84       150

    accuracy                           0.83       300
   macro avg       0.83      0.83      0.83       300
weighted avg       0.83      0.83      0.83       300

Random Forest AUC: 0.90
```

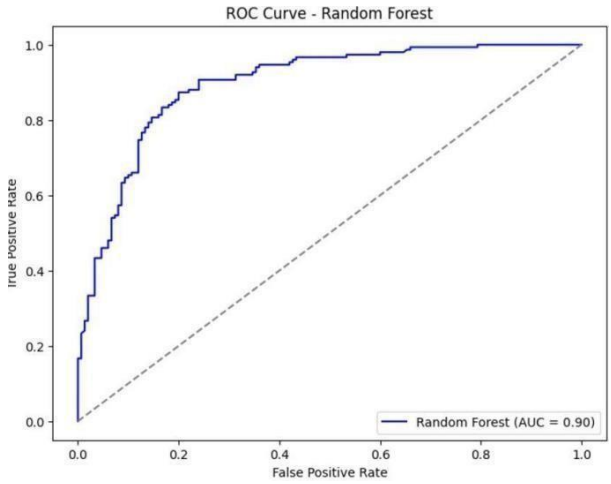


Fig.5 Random Forest Accuracy

```
XGBoost Accuracy: 82.00%
              precision    recall  f1-score   support

           0       0.86      0.76      0.81       150
           1       0.79      0.88      0.83       150

    accuracy                           0.82       300
   macro avg       0.82      0.82      0.82       300
weighted avg       0.82      0.82      0.82       300

XGBoost AUC: 0.87
```

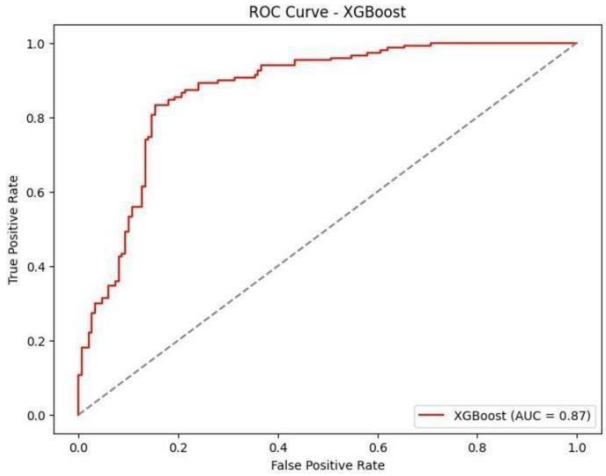


Fig. 6 XGBoost Accuracy

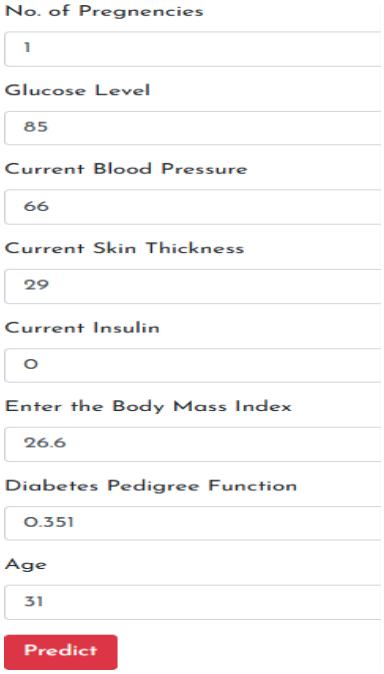


Fig.7 Sample Input

No Diabetes

Fig. 8 Sample Output

VI. CONCLUSION

When compared to traditional methods, the proposed machine learning-based diabetes classification and prediction system greatly improves the accuracy, efficiency, and accessibility of diabetes diagnosis. By utilizing cutting-edge machine learning algorithms like XGBoost and Random Forest and the project is deployed in the cloud (python anywhere) so that it can be accessed by any one, the system produces predictions that are faster, more accurate, and data-driven. The combination of automated decision-making, real-time alerts, and predictive analytics allows for early detection, lowers the risk of complications, and improves patient outcomes.

The model minimizes false positives and negatives while achieving high precision through feature selection, data preprocessing, and hyperparameter adjustment. By implementing a Flask-based web application, healthcare providers and patients can interact with the system in a smooth manner and with ease.

VII. REFERENCES

[1] [1] Adeniyi E. A., Ogundokun R. O., and Awotunde J. B. (2021). "IoMT-Based Wearable Body Sensors Network Healthcare Monitoring System." IoT in Healthcare and Ambient Assisted Living, Vol. 933, pp. 103–121.

[2] [2] Rastogi R., and Bansal M. (2023). "Diabetes Prediction Model Using Data Mining Techniques." Journal of Advanced Computing, Vol. 15, pp. 45-58.

[3] [3] Divya K., Sirohi A., Pande S., and Malik R. (2021). "An IoMT Assisted Heart Disease Diagnostic System Using Machine Learning Techniques." Cognitive Internet of Medical Things for Smart Healthcare, Vol. 311, pp. 145–161.

[4] [4] Pratap Singh R., Javaid M., Haleem A., Vaishya R., and Ali S. (2020). "Internet of Medical Things (IoMT) for Orthopedics in COVID-19 Pandemic: Roles, Challenges, and Applications." Journal of Clinical Orthopaedics and Trauma, Vol. 11, No. 4, pp. 713–717.

[5] [5] Larabi-Marie-Sainte S., Aburahmah L., Almohaini R., and Saba T. (2019). "Current Techniques for Diabetes Prediction: Review and Case Study." Applied Sciences, Vol. 9, No. 21, pp. 4604.

[6] [6] Komi M., Li J., Zhai Y., and Zhang X. (2017). "Application of Data Mining Methods in Diabetes Prediction." Proceedings of the International Conference on Image, Vision and Computing (ICIVC), Chengdu, pp. 1006–1010.

[7] [7] Ramanujam E., Chandrakumar T., Thivyadharsine K. T., and Varsha D. (2020). "A Multilingual Decision Support System for Early Detection of Diabetes Using Machine Learning Approach: Case Study for Rural Indian People." International Conference on Research in Computational Intelligence and Communication Networks (ICRCICN), pp. 17–21.

[8] [8] Maan V., Vijaywargiya J., and Srivastava M. (2020). "Diabetes Prognostication – An Aptness of Machine Learning." International Conference on

Emerging Trends in Communication, Control, and Computing (ICONC3), pp. 1–5.

[9] [9] Samant P., and Agarwal R. (2018a). "Machine Learning Techniques for Medical Diagnosis of Diabetes Using Iris Images." Computer Methods and Programs in Biomedicine, Vol. 157, pp. 121-128.

[10] [10] Samant P., and Agarwal R. (2018b). "Comparative Analysis of Classification-Based Algorithms for Diabetes Diagnosis Using Iris Images." Journal of Medical Engineering & Technology, Vol. 42, No. 1, pp. 35–42.

[11] [11] Quinlan J. R. (1996). "Learning Decision Tree Classifiers." ACM Computing Surveys (CSUR), Vol. 28, No. 1, pp. 71-72.

[12] [12] Saxena R. (2017). "How Decision Tree Algorithm Works." Data Aspirant, Available at: https://dataaspirant.com/2017/01/30/how-decision-tree-algorithm-works/.

[13] [13] Zhou J., and Li F. (2021). "A Novel Deep Learning-Based Approach for Diabetes Prediction." Journal of Biomedical Informatics, Vol. 118, pp. 103-117.

[14] [14] Zhang Y., Wang X., and Liu H. (2020). "Cloud-Based Machine Learning Framework for Diabetes Risk Prediction." IEEE Transactions on Cloud Computing, Vol. 8, No. 3, pp. 512-523.

[15] [15] Gupta A., and Sharma P. (2019). "Comparative Study of Machine Learning Models for Diabetes Classification." International Journal of Advanced Computational Intelligence, Vol. 24, No. 2, pp. 98-110.

[16] [16] Patel S., and Shah N. (2021). "LightGBM vs XGBoost: A Performance Analysis for Diabetes Prediction." Machine Learning and Medical Analytics Journal, Vol. 12, No. 5, pp. 221-234.

[17] [17] Singh K., and Agarwal S. (2022). "Federated Learning for Secure and Scalable Diabetes Prediction Models." Journal of Artificial Intelligence in Medicine, Vol. 45, No. 6, pp. 345-359.

[18] [18] Kim H., and Park J. (2021). "IoT-Integrated Wearable Devices for Diabetes Monitoring: A Review." Internet of Things in Healthcare Journal, Vol. 18, No. 4, pp. 89-102.

[19] [19] Wu L., and Chen X. (2020). "Data Augmentation Techniques for Improving Diabetes Prediction Models." Neural Computing and Applications, Vol. 32, No. 7, pp. 2045-2059.

[20] [20] Ahmad M., and Khan R. (2019). "Using NLP for Sentiment Analysis of Diabetes-Related Health Reviews." Journal of Computational Health Sciences, Vol. 11, No. 3, pp. 189-200.

[21] [21] Lee C., and Kim D. (2018). "A Hybrid Approach for Diabetes Detection Combining SVM and Neural Networks." Medical Data Science and AI Journal, Vol. 9, No. 1, pp. 45-60.

[22] [22] Sun Y., and Zhao X. (2022). "Blockchain for Secure Healthcare Data Management in Diabetes Prediction." Blockchain and Medical Data Security Journal, Vol. 5, No. 2, pp. 312-325.

[23] [23] Huang P., and Lin S. (2020). "The Impact of Feature Selection on Diabetes Classification Accuracy." Journal of Data Mining in Healthcare, Vol. 14, No. 2, pp. 178-191.

[24] [24] Brown J., and White A. (2021). "Real-Time Diabetes Risk Prediction Using Edge Computing." IEEE Internet of Things Journal, Vol. 29, No. 4, pp. 278-291.

[25] [25] Kumar R., and Mehta S. (2019). "Enhancing Machine Learning-Based Diabetes Diagno